

Technology
Arts Sciences
TH Köln

Cologne University of Applied Sciences

Faculty of Information, Media and Electrical Engineering

Bachelor's Thesis in Mediatechnology

Realtime Dereverberation Using Deep Learning

Echtzeit-Enthallung auf Basis von Deep-
Learning-Verfahren

Authors:	Leo Kling, Jonathan Kron
Submitted to:	Prof. Dr. Christoph Pörschmann
Second reviewer:	Prof. Dr. Jan Salmen
Start Date:	12.01.2026
Submission Date:	13.04.2026

We hereby confirm that this Bachelor's thesis is our own work and we have documented all sources and material used.

Cologne, 13.04.2026

Leo Kling & Jonathan Kron

Transparency in the use of AI tools

Coding Assistance

- Microsoft Copilot
- OpenAI ChatGPT 5.2 & 5.3-Codex
- Claude Sonnet 4.6

Writing Assistance

- DeepL for translations

Acknowledgements

For Gondor!

Computations were performed with computing resources granted by RWTH Aachen University under project **thes2191**.

Abstract

Note:

1. **paragraph:** What is the motivation of your thesis? Why is it interesting from a scientific point of view? Which main problem do you like to solve?
2. **paragraph:** What is the purpose of the document? What is the main content, the main contribution?
3. **paragraph:** What is your methodology? How do you proceed?

Zusammenfassung

Note: Insert the German translation of the English abstract here.

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Problem	2
1.3	Objectives	2
1.4	Background	3
2	Related Work	5
2.1	Conv-TasNet	5
2.2	StoRM	6
2.3	DeepFilterNet	7
2.4	Quality Net	7
3	Theoretical Background	10
3.1	Acoustics	10
3.1.1	Reverberation	10
3.1.2	Reverberation in Active Acoustics Systems	10
3.2	Neural Networks	10
3.2.1	Architectures	10
3.2.1.1	CNN	10
3.2.1.2	RNN	11
3.2.1.3	TCN	12
3.2.1.4	Autoencoders	14
3.2.2	Supervised and Self-Supervised Learning	16
3.2.3	Training of a Neural Network	17
3.2.3.1	Loss Function	17
3.2.3.2	Backpropagation and Autograd	18
3.2.3.3	Gradient Descent	21
3.2.3.4	Taxonomy of Loss Functions	21
3.3	Quality Metrics	22

3.3.1	MAE and MSE	23
3.3.2	Correlation	23
3.3.3	SI-SNR	24
3.3.4	PESQ	24
3.3.5	PEAQ	25
4	Methodology	27
4.1	Dataset	27
4.1.1	Data Collection	28
4.1.2	Dataset Creation	29
4.1.2.1	Reverberation	30
4.1.2.2	Calculation of PEAQ Scores	32
4.1.2.3	Non-Silent Parts	32
4.2	Analyzation of Applicable Loss Functions	33
4.3	Perceptual Quality Network	37
4.4	Objective Quality Network	39
5	Implementation & Experimental Setup	40
5.1	Conv-TasNet for diverse audio dereverberation	40
5.2	Perceptual Quality Network	41
5.2.1	Simple CNN	41
5.2.2	CNN14	43
5.3	Objective Quality Network	45
5.4	Dereverberation Network	45
5.5	Placeholders	46
5.5.1	SOMETHING WITH SEGMENT LENGTH	46
5.5.2	LOSS FUNCTION SILENT MASK	46
6	Results	47
6.1	Conv-TasNet	47
6.2	StoRM	50
6.3	Perceptual Quality Network	51
6.4	Objective Quality Network	52

6.5 Dereverberation Network	54
7 Evaluation	56
7.1 Perceptual Quality Net	56
7.1.1 Simple CNN	56
7.1.2 CNN14	57
7.2 Objective Quality Net	58
7.3 Comparison of Conv-TasNet and StoRM for diverse signals	58
7.4 DISCUSS UPSAMPLING FOR TRAINING AND REVERBERATING AT LOWER (USING PLOTS)	61
7.5 (An)echoic dataset	61
7.6 SI-SNR Calculations	61
8 Conclusion	62
List of Figures	63
List of Tables	65
Appendix A: Supplementary Material	66
Bibliography	67

Glossary

Acoustics

AIR. Aachen Impulse Response	29
ASR. automatic speech recognition	1, 43
DCASE. Detection and Classification of Acoustic Scenes and Events	27
RIR. room impulse response	29, 30, 31
STT. speech-to-text	1

Metrics

BSD. Bark Spectral Distortion	24
DI. distortion index	26, 32, 36
KDE. kernel density estimation	35, 36, 37
LCC. linear correlation coefficient	8, 9
LMS. least mean squares	18
MAE. mean absolute error	8, 23, 37, 56, 57
MOS. mean opinion score	25, 38, 65
MSE. mean squared error	8, 2, 9, 23, 24, 33, 37, 42, 47, 48, 49, 56, 57, 63
MSS. multi-scale spectral loss	47, 48, 63
ODG. objective difference grade	26, 32, 36, 38, 41, 45, 57, 59
PAQM. Perceptual Audio Quality Measure	25
PEAQ. Perceptual Evaluation of Audio Quality	8, 25, 26, 32, 34, 38, 39, 63
PESQ. Perceptual Evaluation of Speech Quality	8, 2, 9, 24, 25, 33, 34, 36, 38, 48, 59, 63, 65

PSQM. Perceptual Speech Quality Measure	24
SDG. subjective difference grade	26
SI-SDR. scale-invariant signal-to-distortion ratio	48
SI-SNR. scale-invariant signal-to-noise ratio	8, 2, 5, 24, 33, 36, 37, 47, 48, 49, 56, 59
SRCC. Spearman's rank correlation coefficient	8, 9
ViSQOL. Virtual Speech Quality Objective Listener	34

Neural Networks

BLSTM. bidirectional long short-term memory	8, 12
CNN. convolutional neural network	7, 8, 9, 3, 10, 11, 41, 56, 57, 65
CV. computer vision	27, 30
DAE. denoising autoencoder	14, 15, 63
GRU. gated recurrent unit	12
LLM. large language model	27
LSTM. long short-term memory	12
RNN. recurrent neural network	7, 8, 10, 11, 12, 13, 63
ReLU. rectified linear unit	19, 38, 41, 57
SGD. stochastic gradient descent	21
TCN. temporal convolutional network	7, 5, 10, 12, 13, 14, 45

Signal Processing

ERB. equivalent rectangular bandwidth	7
STFT. short-time Fourier transform	3, 4, 7, 41

1 Introduction

Reverberation is apparent in most audio signals as it is an inherent characteristic of recording environments. It is caused by late reflections ($> 50 - 80$ ms) that overlap with the direct sound in the diffuse sound field [Kuh25]. Studies have shown that reverberation is an important auditory cue which informs the listener over environmental factors [TM16]. Depending on the application reverberation can be an attractive addition to the auditory signal, such as in music [Nay14] or speech performances.

The inverse task aptly named dereverberation, is a process of removing reverberant parts of a recorded audio signal under reverberant conditions. Thus retaining only the direct sound of the recording. This was only made feasible due to recent advancements in the field of deep learning. Historically, this task was approached using standard signal processing techniques that involved coherence estimation, suppression rules based on thresholds and gain functions [ABB77, BC82].

In many modern applications, dereverberation is highly desirable. We divide use cases into two main categories: *offline* and *live* processing. Offline applications do not strictly require real-time operation, although real-time capability may still be beneficial.

1.1 Motivation

Studies have shown that the adverse effects of reverberation mainly the temporal smearing of target speech and background noise, masking, coloration and signal-to-noise degradation [CLG22, Fig+25, Kuh25] significantly degrade human as well as automatic speech recognition (ASR) or STT [Neu+10, Pug+21]. These effects also negatively affect the overall quality of diverse audio signals such as in music remixes and film post-production, where excessive room reverberation can reduce audio clarity and limit creative flexibility. While the above named offline applications are not in need of real-time processing, live applications, as they are used in interactive scenarios such as video conferencing, speech recognition systems,

and live music performance impose strict constraints on processing latency and computational efficiency.

1.2 Problem

Artificial reverberation can be added to audio signals with comparatively simple signal processing techniques, the inverse task of removing or reducing existing reverberation is significantly more complex [Att+00]. Reverberation is a time-dispersive and highly non-linear process, where direct sound and multiple delayed reflections overlap in both time and frequency [Dat97]. This overlap makes a clear separation between the original (dry) signal and the reverberant components (wet) difficult and, for a long time, was considered practically unsolvable using classical digital signal processing methods [Bra98].

We address several open challenges in this thesis. First, it is unclear how well established dereverberation architectures generalize to diverse audio signals such as music and mixed content [LM18]. Current dereverberation models utilize different loss functions such as mean squared error (MSE), scale-invariant signal-to-noise ratio (SI-SNR) or Perceptual Evaluation of Speech Quality (PESQ) [Lem+23, LM19, Rad21] whose indication of dereverberation performance in diverse audio signals is not well documented. It is therefore unclear whether these loss functions are applicable for our use case or if other qualitative metrics would improve results. Second, there are time-domain and frequency-domain approaches, which can be investigated in terms of audio quality, computational complexity, and latency. Third, real-time applicability imposes strict latency limits (e.g. below 50 ms) [Sch+24] that strongly influence network architecture, window size, and sampling rate.

1.3 Objectives

The central objective of this thesis is to explore deep-learning-based dereverberation methods. As our primary question we ask whether a model can be designed to operate in real-time while maintaining perceptually convincing audio quality for a wide range of audio signals including music, speech and other noises (e.g. vehicle

or animal sounds). Solutions for multiple performance constraints are surveyed mainly the dataset, domain and loss function.

The dataset processing and curation process is discussed regarding bias, variance and sample quality.

Time-domain and frequency-domain neural network approaches are investigated by evaluating their qualitative performance and analyzing their suitability for low-latency, real-time applications through comparison of domain specific literature and our own neural network implementations.

The performance of different quality metrics is assessed and their qualitative performance for use as loss functions examined leading to our own implementation of a loss neural network.

1.4 Background

As mentioned in Section 1.2, dereverberation is a problem that has recently been addressed using machine learning methods. Three main approaches have emerged, each with their own strenghts and shortcomings.

Classic signal processing implementations aim to enhance audio through subtraction of a predicted noisy signal [Bol79] or Wiener filtering [Mad+13]. Further developing these methods has introduced deep filtering approaches, utilizing machine learning to optimize frequency filter paramters [Zha+21]. Section 2.3 examines the “DeepFilterNet” model.

Other literature can be devided into time-domain or frequency-domain approaches, based on the representation of audio used. Time-domain methods operate directly on the audio waveform and use individual samples as input into the network. Frequency-domain approaches on the other hand transform the input waveform into a frequency-domain representation first, commonly using the short-time Fourier transform (STFT).

This has the advantage of being able to rely on convolutional neural network (CNN) architectures, which allow for faster processing and lower parameter counts and preservation of local correlations in time and frequency that can be

captured efficiently by two-dimensional filters (see Section 3.2.1.1). Time-domain approaches allow for longer temporal contexts, allowing the model to better understand temporal dependencies (see Section 3.2.1.2 and Section 3.2.1.3).

The two models compared, as well as the one presented in this thesis, can be classified in the latter two categories. StoRM [Lem+23] is a frequency-domain model, making use of the STFT, while Conv-TasNet [LM19] and our novel implementation both operate in the time-domain.

Recent literature regarding dereverberation and adjacent problems (e.g. speaker–source separation and denoising) has focused on improving model architectures and loss functions for better performance. Notably Fu et al. (2021) utilize a novel neural network based loss for qualifying source separation performance introduced by Fu et al. (2018). This approach is further discussed in Section 2.4 and has motivated our own implementation of a perceptual quality loss network (see Section 4.3).

2 Related Work

2.1 Conv-TasNet

Leo Kling

Conv-TasNet extends the time-domain audio separation paradigm introduced by TasNet [LM18]. TasNet was proposed as an alternative to short-time Fourier transform based source separation, motivated by the phase–magnitude decoupling of spectral methods, the fixed analysis representation, and the latency introduced by time-frequency decomposition. Instead of predicting masks on a spectrogram, TasNet learns an encoder–decoder representation directly on the waveform and performs separation in this latent space. Conv-TasNet retains this general idea, but replaces the original recurrent separator with a fully convolutional architecture designed to model long temporal context more efficiently [LM19].

Architecturally, Conv-TasNet consists of a learned linear encoder, a masking network, and a linear decoder. The encoder transforms the input waveform into a latent representation, the separator estimates multiplicative masks for this representation, and the decoder reconstructs the target waveform from the masked features. The key architectural contribution of Conv-TasNet is the use of a temporal convolutional network (TCN) with stacked dilated one-dimensional convolution blocks as the separation module (see Section 3.2.1.3). This allows the model to capture long-range temporal dependencies while remaining compact and fully convolutional. In the original paper, the model is trained with SI-SNR and evaluated on single-channel, speaker-independent speech separation, where it outperforms preceding time-frequency masking approaches and even surpasses ideal time-frequency magnitude masking on the WSJ0 benchmark [LM19].

For this thesis, Conv-TasNet is relevant because it combines strong reported speech-domain performance with low model complexity and a comparatively small minimum latency, making it a plausible candidate for real-time capable dereverberation systems. At the same time, the scope of the original work remains limited to speech separation. The paper does not investigate dereverberation on diverse broadband material such as music, environmental sounds, or mixed acoustic

scenes. Therefore Conv-TasNet here serves not as a solved answer to the research problem, but as a strong real-time speech-domain baseline whose transferability to diverse-signal dereverberation must be evaluated separately.

2.2 StoRM

Leo Kling

StoRM is a diffusion-based stochastic regeneration model for speech enhancement and dereverberation [Lem+23]. In contrast to purely predictive enhancement systems, it follows a fully generative formulation in which the target signal is refined through a reverse diffusion process. The model combines a predictive estimate with stochastic generative refinement, aiming to retain the robustness of predictive methods while benefiting from the higher sample quality often associated with diffusion-based generation.

Architecturally, StoRM relies on score-based estimation during the reverse diffusion process and uses a predictive model as a guide for the generative reconstruction. In this sense, the score estimation component is conceptually related to the scoring used in this thesis, although it serves a different role than the perceptual loss network introduced later. The main motivation of the original paper is to reduce artifacts that may arise in purely generative diffusion systems while still producing very clean speech restoration results. At the same time, the paper explicitly notes the high computational burden of diffusion-based inference, since multiple reverse steps are required instead of a single forward pass.

For this thesis, StoRM is relevant because it demonstrates that a generative speech-domain approach can achieve strong dereverberation quality under its intended conditions. However, the scope of the original work remains limited to speech signals and a 16 kHz setting, which limits the representable spectrum to 8 kHz by the Nyquist theorem, which is substantially below the upper range of human hearing and therefore does not preserve the full audible bandwidth of music and other broadband audio material [Aco23, Sha49, Zwi61]. In addition, the model is not designed around strict real-time constraints, making it less suitable as a direct answer to the low-latency objective of this thesis. StoRM therefore serves here not as a solved solution to real-time dereverberation of diverse audio, but

as a high-quality speech-domain reference whose transferability to broader signals and latency-constrained applications must be evaluated separately.

2.3 DeepFilterNet

Jonathan Kron

DeepFilterNet is a two stage speech enhancement framework utilizing deep filtering [Sch+22]. Deep filtering is based on the idea that a neural network estimates a complex mask which is applied to the STFT representation of a signal. With an appropriate loss function this mask can be learned to perform source extractions or other tasks like dereverberation [MH20].

DeepFilterNet expands on that idea: “Instead of using a complex mask that is applied per TF-bin, [...] a combination of real-valued gains and a deep filter enhancement component [is used].” [Sch+22]. Given a noisy signal DeepFilterNet first transforms it into the frequency-domain using the STFT. Sampling rates up to 48 kHz and window sizes between 5 ms and 30 ms are supported. Using the frequency-domain representation equivalent rectangular bandwidth (ERB) features mimicing human perception are computed. Both the complex features of the STFT and the ERB features are used as inputs for the model. A mask is trained on the complex STFT values and an encoder–decoder network is trained on the ERB features which is later used to weight the complex mask. DeepFilterNet takes advantage from the fact that noise as well as speech usually have a smooth spectral envelope. This allows for a computationally cheap encoder–decoder network.

For this thesis, DeepFilterNet is relevant because it shows how low complexity speech enhancement can be implemented in the frequency-domain. Regarding audio sample rate the scope of this project even superceeds ours but latency is still an issue as window sizes smaller than 5 ms are not supported. As the architecture of DeepFilterNet exploits the spectral properties of speech, adaptation for use with diverse audio is difficult.

2.4 Quality Net

Jonathan Kron

Most speech or audio assessment tools need a reference signal. This requirement “considerably restricts the practicality of such assessment tool” [Fu+18]. Fu et al.

(2018) introduce an end-to-end, non-intrusive speech quality evaluation model named quality net, that aims to predict PESQ scores without a reference signal.

Architecturally Quality-Net is based on bidirectional long short-term memory (BLSTM), which is an improvement to standard recurrent neural networks (RNNs) (cf. Section 3.2.1.2), where sequential data is processed in both forward and backward directions using two separate hidden layers (cf. Figure 1).

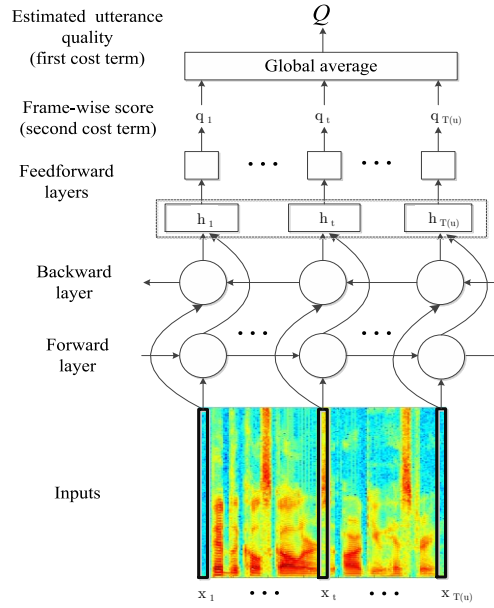


Figure 1: Proposed Quality-Net for end-to-end, non-intrusive speech quality assessment, taken from Fu et al. (2018).

Quality-Net was trained using 4620 utterances from the TIMIT [Gar+93] database. Utterances were divided into clean, noisy and enhanced sets to learn different speech conditions. Noisy signals were generated by corrupting the original speech using 90 different noise types with signal-to-noise levels ranging from -10 dB to 25 dB. The enhanced utterances were first corrupted then enhanced using a BLSTM based enhancement model.

Quality-Net’s performance was evaluated over 100 utterances using the MSE, linear correlation coefficient (LCC) and Spearman's rank correlation coefficient (SRCC) [Fu+18].

	MSE	LCC	SRCC	Variance of frame quality in clean speech
without constraint	0.1441	0.8559	0.8607	4.1128
with constraint	0.1266	0.8749	0.8807	0.2468

Table 1: Results of Quality-Net and the two-stage model replicated from Fu et al. (2018).

A constraint applies conditional weighting to each frame. This was done because noise can vary throughout an audio sample and using a singular quality value for each frame of an audio sample could degrade model performance.

Table 1 shows good correlation but a considerably high MSE value as PESQ values lay between 0 and 4. These findings among other things motivated our own development of a audio assessment model which is discussed in Section 4.3.

3 Theoretical Background

3.1 Acoustics

3.1.1 Reverberation

convolutions are a fast operation in the frequency domain and on GPU devices [MNT16, Sid20]

3.1.2 Reverberation in Active Acoustics Systems

- explain the theoretical background of coloration artifacts in live systems utilizing active acoustics when reverberation is present in a feedback loop

In these cases, reverberation can significantly degrade speech intelligibility, introduce unwanted coloration, and negatively affect the overall user experience [Neu+10, Pug+21].

3.2 Neural Networks

Neural networks are parameterized function approximators composed of interconnected layers of simple computational units. During training, their weights and biases are iteratively adjusted so that the network maps an input to a desired output through minimizing a loss function [GBC16]. In practice, modern architectures differ mainly in how they organize these computations and which inductive bias they impose on the data. For this thesis, the most relevant families are CNNs, RNNs, and temporal convolutional networks (TCNs).

3.2.1 Architectures

Leo Kling

3.2.1.1 CNN

CNNs are feed-forward neural networks that process structured inputs by applying learned convolution kernels over local neighborhoods. Instead of connecting every input element to every neuron, a convolutional layer reuses the same filter weights

across the full input. This weight sharing reduces the number of parameters and makes the network particularly effective at detecting local patterns such as edges in images, harmonics in spectrograms, or short waveform structures [GBC16]. Stacking multiple convolution layers increases the receptive field, so deeper layers can combine simple local features into more abstract representations.

In audio machine learning, CNNs are widely used on time-frequency representations because spectrograms preserve local correlations in time and frequency that can be captured efficiently by two-dimensional filters. One-dimensional variants are also common when operating directly on waveforms or learned feature sequences. Their main advantages are computational efficiency, parameter sharing, and good parallelizability on modern hardware. A limitation is that long-range temporal dependencies are not represented explicitly and must instead be captured through depth, pooling, or large receptive fields [Gra+25, Kon+20].

3.2.1.2 RNN

RNNs are designed for sequential data. In addition to the current input, each recurrent step receives a hidden state that carries information from previous time steps, allowing the network to model temporal dependencies. This makes RNNs conceptually well suited for signals, text, and time series, where the interpretation of one sample often depends on earlier context, [GBC16, RHW86]. In audio applications, recurrent layers have been used for tasks such as speech enhancement, quality prediction, and sequence modeling because they can aggregate information over longer temporal spans than shallow feed-forward models.

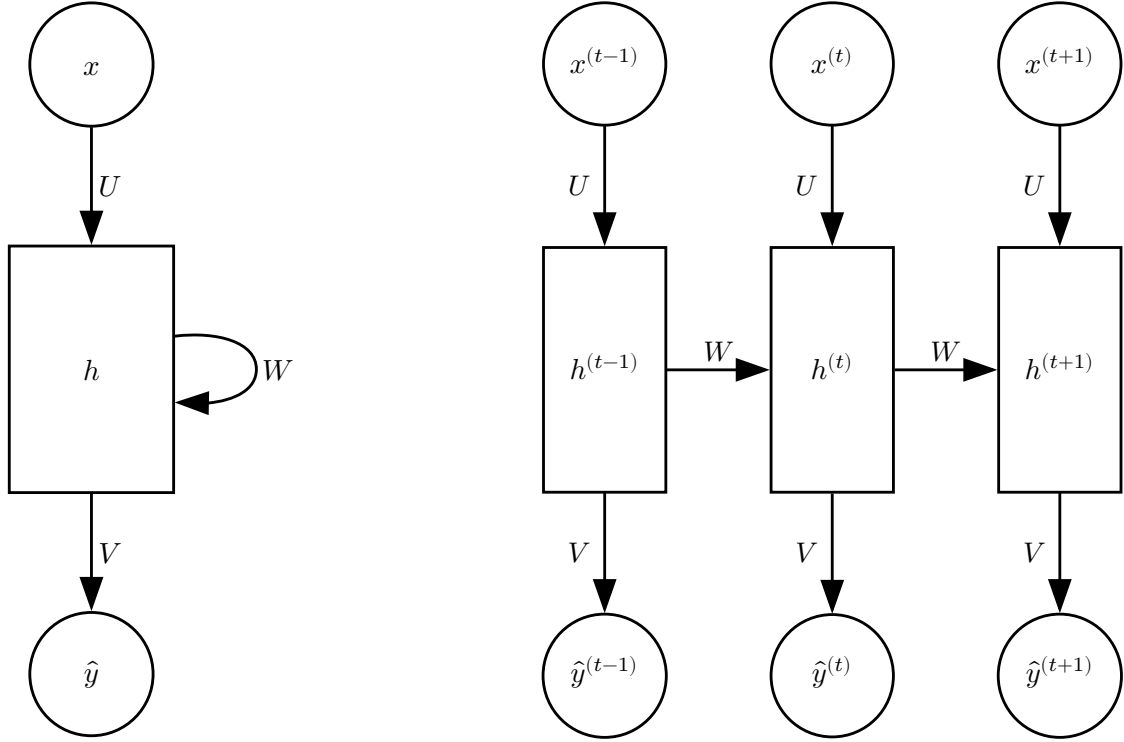


Figure 2: RNN architecture for an input sequence x , hidden connections parametrized by a weight matrix U and hidden-to-hidden recurrent connections parametrized by a weight matrix W . Shown on the right is the unrolled version of an RNN cell across three time steps. Replicated from Goodfellow et al. (2016).

Classical RNNs suffer from vanishing and exploding gradients when the dependency horizon becomes long. For this reason, gated variants such as long short-term memory (LSTM) [HS97], BLSTM and gated recurrent unit (GRU) networks [Cho+14] are commonly used in practice. These architectures regulate which information is stored, updated, or forgotten, improving training stability and long-term memory. Their main drawback is that recurrent processing is inherently sequential, which limits parallelization during training and inference compared to purely convolutional models [D  f+19, Fu+18].

3.2.1.3 TCN

TCNs adapt the convolutional idea to sequence modeling by applying one-dimensional convolutions along the temporal axis. To cover long contexts efficiently, they often use dilated convolutions, where filter taps are spaced apart by increasing

dilation factors. This enlarges the receptive field without requiring very deep networks or large kernels. Depending on the application, TCNs can be implemented causally, where each output depends only on the present and past, or non-causally, where future context is also available [BKK18].

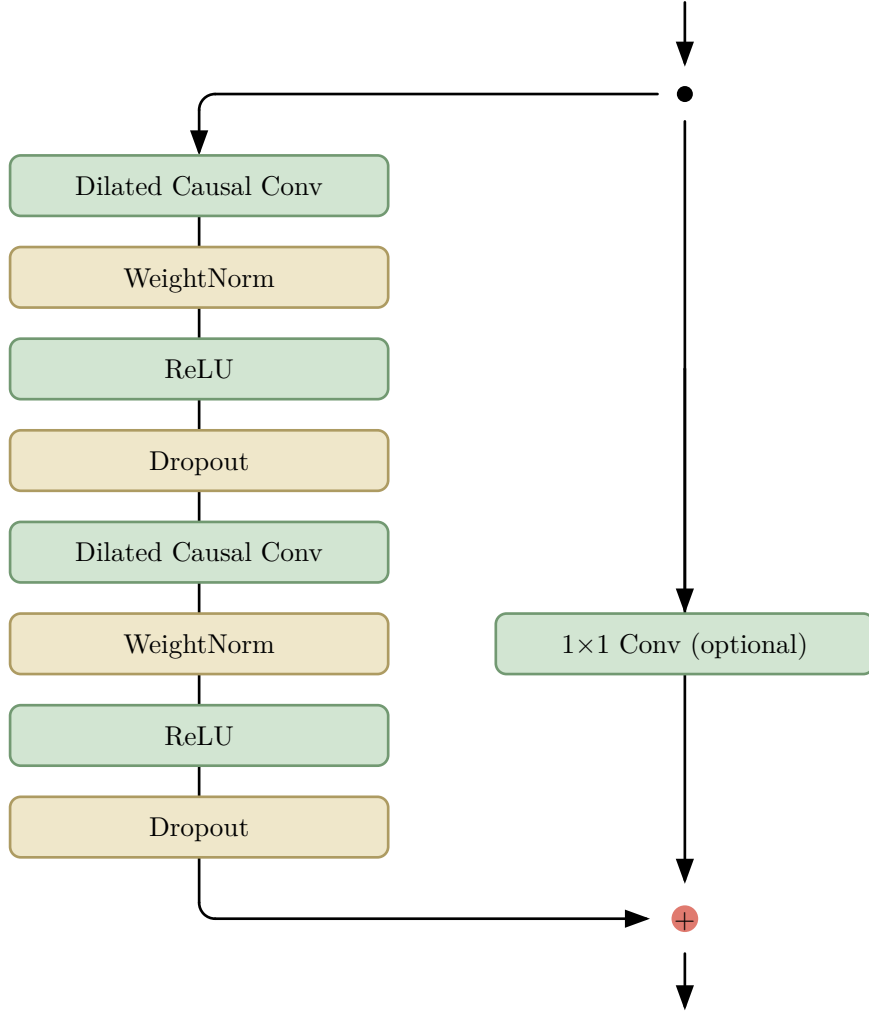


Figure 3: TCN residual block, where the 1×1 Convolution is only added when input and output differ in dimensions. Replicated from Bai et al. (2018).

Compared to RNNs, TCNs retain the ability to model long temporal structure while remaining fully convolutional and therefore highly parallelizable. They also provide explicit control over receptive field size through kernel width, depth, and dilation schedule. This makes them attractive for audio tasks that require a compromise between temporal context and computational efficiency. In this

thesis, TCNs are particularly relevant because Conv-TasNet uses a temporal convolutional network to estimate masks over an encoded waveform representation [LM19]. The basic principles of TCNs therefore form part of the architectural foundation for the dereverberation models discussed later.

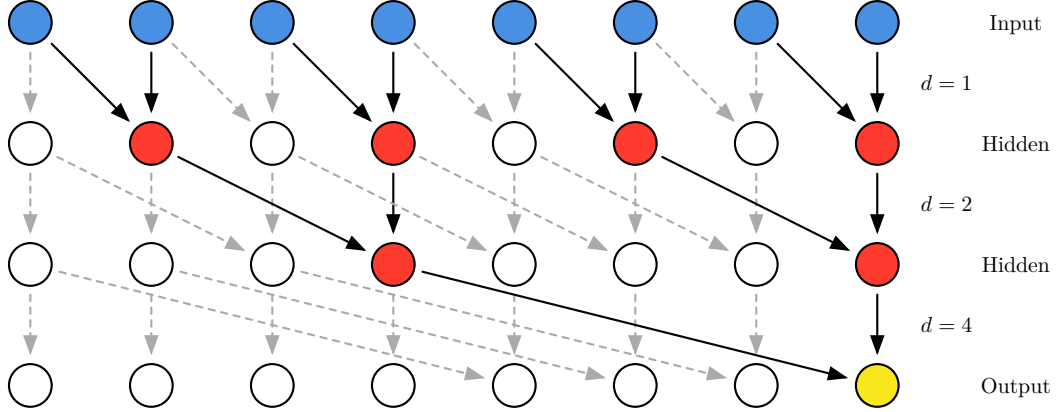


Figure 4: A dilated causal convolution with dilation factors $d = 1, 2, 4$. Replicated from Lee et al. (2021).

3.2.1.4 Autoencoders

Autoencoders try to convert an input vector into a code vector of lower dimensionality and then reconstruct an approximation of the original input from this code vector [HZ93]. This network can be split into two parts, an encoder function

$$\mathbf{h} = f(\mathbf{x}) \quad (3.1)$$

and a decoder

$$\mathbf{r} = g(\mathbf{h}) \quad (3.2)$$

generating a reconstruction $\mathbf{r}$ of the original input $\mathbf{x}$ through a hidden layer $\mathbf{h}$ [LF87, GBC16].

Of particular interest for this thesis are denoising autoencoders (DAEs) which are a type of regularized autoencoder. Instead of minimizing some loss function

$$L(\mathbf{x}, g(f(\mathbf{x}))) \quad (3.3)$$

, where L is some loss function penalizing the difference between the input and the reconstruction, the denoising autoencoder minimizes

$$L(\mathbf{x}, g(f(\tilde{\mathbf{x}}))) \quad (3.4)$$

, where $\tilde{\mathbf{x}}$ is a stochastically corrupted version of the input. This forces the network to learn a more robust representation of the data, as it must be able to undo the corruption from a noisy version [GBC16]. In the context of dereverberation, DAEs can be used to learn a mapping from reverberant to clean speech by treating the reverberant signal as a corrupted version of the clean signal. The encoder learns to extract relevant features that are invariant to reverberation, while the decoder reconstructs the clean signal from these features.

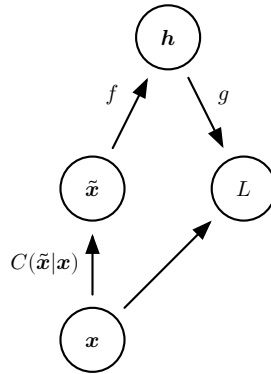


Figure 5: DAE training procedure. Replicated from Goodfellow et al. (2016).

Figure 5 shows the training procedure of a DAE. $C(\tilde{\mathbf{x}}|\mathbf{x})$ is the corruption process which gives a distribution over corrupted versions $\tilde{\mathbf{x}}$, given an original input $\mathbf{x}$. The loss function L is then minimized with respect to the parameters of the encoder f and decoder g from training pairs $(\mathbf{x}, \tilde{\mathbf{x}})$ [GBC16].

talk about skip connections

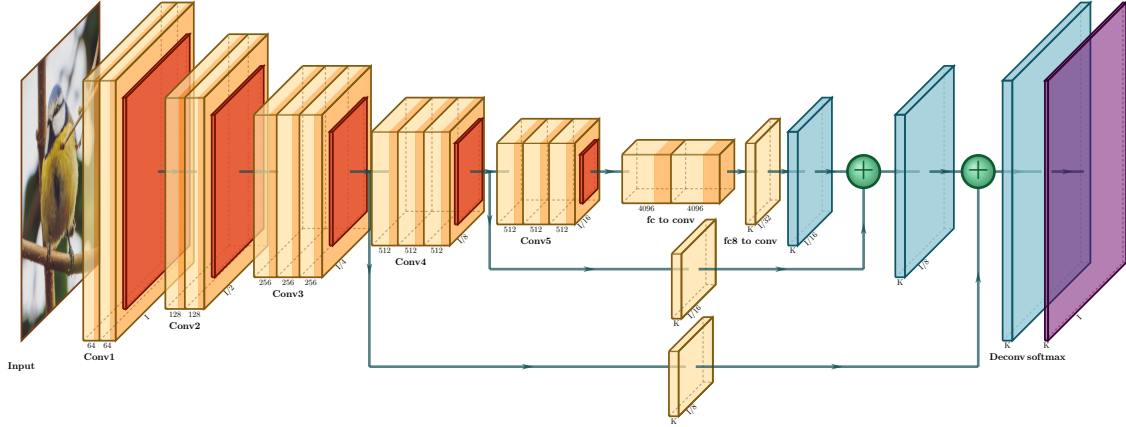


Figure 6: A U-Net architecture (FCN-8)

cite

with skip connections. The left side is the downsampling path where the input is processed through a series of convolutional and pooling layers, while the right side is the upsampling path where the feature maps are upsampled and combined with corresponding feature maps from the downsampling path through skip connections.

3.2.2 Supervised and Self-Supervised Learning

Leo Kling

Training data can be given to a network in different ways. The most common way is to use supervised learning where each input data point is paired with a target output. The network learns to map inputs to their corresponding targets by minimizing a loss function that quantifies the error between the predicted output and the true target. We call this process **supervised learning**, because one can imagine a supervisor who shows the network the correct answer for each input and the network learns to mimic this behavior [GBC16].

Self-supervised learning is a form of unsupervised learning where, instead of relying on external labels, the data itself is used to generate supervisory signals. This is often done by augmentation of the input data to formulate this signal. Self-supervised learning has been successful in recent years, especially in labeled data scarce domains such as audio processing, as shown by Baevski et al. (2020). However, as our task is a supervised regression from reverberant to dry audio, we cannot classify our approach as self-supervised.

3.2.3 Training of a Neural Network

Jonathan Kron

Section 3.2 states neural networks are iteratively trained through minimizing a loss function. The loss function is parameterized through an input-output function as well as the weights of the model. Optimizing the loss function means optimizing the weights.

We can image a multidimensional error landscape formed by the weights. To traverse this error landscape into a local minimum we use the partial derivative of the loss function, also called gradient. This process coined gradient descent is discussed in Section 3.2.3.3.

This gradient was historically computed analytically (see Section 3.2.3.1). Modern multi-million parameter networks make this approach impossible. To aid the process modern networks use automatic differentiation based on the backpropagation method as introduced by Rumelhart et al. (1986).

3.2.3.1 Loss Function

A loss function, also called cost function, is a qualitative function that is used to objectively measure model performance by calculating the deviation of the model's prediction to their ground truth counterpart. This deviation is mapped onto a real number that intuitively represents some error.

To introduce the application of loss functions we want to discuss one of the earliest and simplest neural networks called Adaline [Wid60]. This single-layer neural network defines its input-output function as:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{n=1}^N x_n w_n + b \quad (3.5)$$

where $\mathbf{x}$ is the input vector, $\mathbf{w}$ the weight vector, N the number of inputs, b some bias and y the model output. Assuming that $x_0 = 1$ and $w_0 = b$ the output is simplified to:

$$y(\mathbf{x}, \mathbf{w}) = \sum_{n=1}^N x_n w_n \quad (3.6)$$

. Adaline uses the least mean squares (LMS) algorithm to define its loss, also called cost function:

$$C(d, y) = (d - y(\mathbf{x}, \mathbf{w}))^2 \quad (3.7)$$

where d is the desired target. For analytical simplicity the loss function is often denoted as:

$$\begin{aligned} C(d, y) &= \frac{1}{2}(y(\mathbf{x}, \mathbf{w}) - d)^2 \\ &= \frac{1}{2}(x_1 w_1 + x_2 w_2 + \dots + x_n w_n - d)^2 \end{aligned} \quad (3.8)$$

. The partial derivative, also called gradient, can be calculated analytically (here for the first weight) by deriving the input-output function:

$$\begin{aligned} \frac{\partial C}{\partial w_1} &= \frac{1}{2} \cdot 2 \cdot (y(\mathbf{x}, \mathbf{w}) - d) \cdot y(\mathbf{x}, \mathbf{w})'_{w_1} \\ &= (y(\mathbf{x}, \mathbf{w}) - d) \cdot x_1 \end{aligned} \quad (3.9)$$

. The learning rule implementing this partial derivative is denoted as:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta(d - y(\mathbf{x}, \mathbf{w}))\mathbf{x} \quad (3.10)$$

where η is some factor called the learning rate. This update rule implements gradient descent for linear regression. It should be noted that y is quadratic in the above loss function. Therefore no local minima are offered and only a global minium is approached [Ama93].

It can be concluded from the example above that analytical derivation of such loss functions becomes near impossible for complex input-output functions featuring non-linearities (activation functions) and millions of parameters. To solve this issue the backpropagation algorithm is used.

3.2.3.2 Backpropagation and Autograd

Training a neural network happens in two steps. Initially the input is run through each of the networks functions. Through this process, called forward propagation, the neural network makes its best guess about the correct output.

Once an input-output pair is computed the neural network calculates the gradient of the error function in regards to its guess by traversing backwards through its layers, collecting the derivatives of the error with respect to the parameters of the functions which are later used to change each weight. This operation is also known as gradient descent (see Section 3.2.3.3).

The following section will discuss backpropagation as introduced by Rumelhart et al. (1986).

Expanding on the network example of Section 3.2.3.1 we define our multi-layer network as having a leftmost layer of input units, any number of intermediate layers and a rightmost layer of output units. Connections within a layer or from right to left are forbidden, but connections can skip intermediate layers. The states of the units in each layer are determined by applying equations Equation (3.11) and Equation (3.12)

$$x_j = \sum_i y_i w_{ji} \quad (3.11)$$

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (3.12)$$

, also called the forward pass, where Equation (3.12) is the sigmoid function which today is often replace by the rectified linear unit (ReLU) activation function.

The total error E is defined as (cf. Equation (3.8))

$$E = \frac{1}{2} \sum_c \sum_j (y_{j,c} - d_{j,c})^2 \quad (3.13)$$

where c is an index over all input-output paris, j is an index over output units, y is the actual state of an output unit and d is the desired state. The backward pass starts by computing the parital derivative of E in respect to x_j for each output unit. Differentiating Equation (3.13) for a single input-output pair

$$\begin{aligned} E &= \frac{1}{2} (y_j - d_j)^2 \\ &= \frac{1}{2} \left(\left(\frac{1}{1 + e^{-x_j}} \right) - d_j \right)^2 \end{aligned} \quad (3.14)$$

by applying the chain rule gives:

$$\begin{aligned}\frac{\partial E}{\partial x_j} &= \frac{\partial E}{\partial y_j} \cdot \frac{dy_j}{dx_j} \\ &= \frac{\partial E}{\partial y_j} \cdot y_j(1 - y_j)\end{aligned}\tag{3.15}$$

. This shows the affecting change is just a linear function of the states of the layer before making it “easy” [RHW86] to compute how the error will be affected by changing states in the intermediate layers. For a weight w_{ji} the derivative is

$$\frac{\partial E}{\partial w_{ji}} = \frac{\partial E}{\partial x_j} \cdot y_i\tag{3.16}$$

The output of the i^{th} unit taking into account all emerging connections results in

$$\frac{\partial E}{\partial y_i} = \sum_j \frac{\partial E}{\partial x_j} \cdot w_{ji}\tag{3.17}$$

. This shows how $\frac{\partial E}{\partial y}$ of the output layer can be computed when $\frac{\partial E}{\partial y_i}$ of the layer before is given. This procedure can therefore be repeated for each layer going backwards.

Historically these computations have been done manually by the researchers [Bay+15]. This task is tedious and error prone. Here automatic differentiation algorithms are of assistance. Pytorch’s autograd system stores all functional computations that create the neural network’s guess in a directed acyclic graph “whose leaves are the input tensors and roots are the output tensors. By tracing this graph from roots to leaves, you can automatically compute the gradients using the chain rule” [Aut26]. It is important to note that this automatic process requires every function to respect the input data’s need for a gradient. During computation gradient calculation can be accidentally disabled. This problem can occur when using another neural network as the loss function. This is further discussed in Section 5.4.

3.2.3.3 Gradient Descent

In Section 3.2.3.2 is discussed how partial derivatives of the error function E can be calculated either manually or through the use of an automatic differentiation system.

Once ∇E is calculated each weight can be adjusted so that the loss is further minimized (cf. Equation (3.10)). Through this process is called gradient descent a local minimum is searched. Finding a global minimum is not necessary, as experts “suspect that, for sufficiently large neural networks, most local minima have a low cost function value, and that it is not important to find a true global minimum” [GBC16].

Rumelhart et al. (1986) introduce the simplest version of gradient descent as the accumulation of all gradients over all training examples and changing each weight by an amount proportional to the accumulated $\frac{\partial E}{\partial w}$. There are in fact improvements to this approach in the stochastic gradient descent (SGD) method which approximates the gradient of the entire dataset over a small subset of training examples also called minibatches. This lowers the computational cost of calculating a gradient over the entire dataset which is especially useful when dealing with large amounts of data. It is not guaranteed that the SGD method arrives at a local minimum in a reasonable amount of time, but often a useful “low enough” loss is found [GBC16].

SGD is used during training of our perceptual quality network as well as the dereverberation network.

3.2.3.4 Taxonomy of Loss Functions

Section 3.2.3.1 and Section 3.2.3.2 made clear what impact a loss function can have on the training process of a neural network. Over the recent years many different loss functions for different problem sets have been envisioned each best suited for a specific input-output function with specific input-output data pairs [Cia+24].

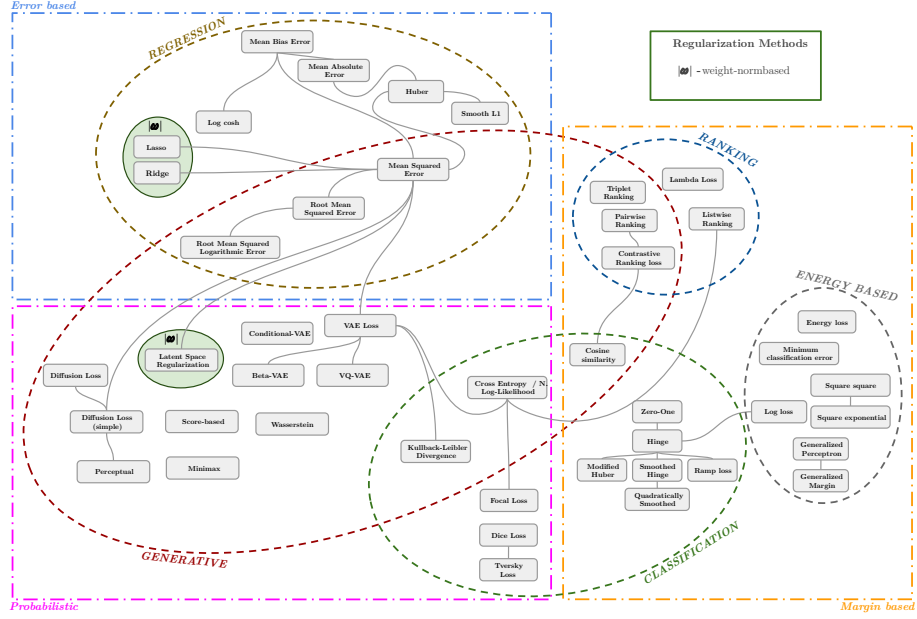


Figure 7: A taxonomy of loss functions taken from Ciampiconi et al. (2024).

Figure 7 shows a map that identifies five major tasks for which loss functions can be designed, namely regression, classification, ranking as well as generative and energy based models. Optimization strategies for each task category are proposed including error-based, probabilistic and margin based loss functions.

The problem of dereverberation can be attributed to a regressive task. It is shown that for the problem of regression, error-based loss functions are applicable. The following section will discuss error-based metrics which can be utilized as loss functions. An analysis of their performance as such is discussed in Section 4.2.

3.3 Quality Metrics

Jonathan Kron

The following section will present different quality metrics desgined for comparative analysis of two input vectors. Going forward the input vectors will be considered signals as we are examining these measures from a signal processing standpoint.

$\mathbf{s}$ is defined as the ground truth, also named reference or true, signal.

$\hat{\mathbf{s}}$ is defined as the predicted, also named test or processed, signal.

All subsequent measures are investigated for general usability in audio adjacent machine learning tasks. Most are used in Chapter 6 for comparative evaluation of different neural networks. A discussion of usability as a loss function for a dereverberation neural network is found in Section 4.3.

3.3.1 MAE and MSE

The mean absolute error (MAE)

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n (s_i - \hat{s}_i) \quad (3.18)$$

measures the average absolute error between to signals. The mean squared error

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (s_i - \hat{s}_i)^2 \quad (3.19)$$

measures the average squared difference between the predicted and the ground truth signal. Although both the MSE and MAE were used successfully as loss functions in e.g. music source separation approaches [Déf+19, SED18, TM20] they fall short in generative and human-ear centered tasks as both unfairly penalize shifts in time and amplitude of the predicted signal and do not conform to the equal-loudness levels as perceived by the human ear [Aco23] and therefore overweight the importance of low frequencies.

3.3.2 Correlation

The Pearson’s product-momentum coefficient is defined as:

$$\rho_{s,\hat{s}} = \text{corr}(s, \hat{s}) = \frac{\text{cov}(s, \hat{s})}{\sigma_s \sigma_{\hat{s}}} = \frac{\text{E}[(s - \mu_s)(\hat{s} - \mu_{\hat{s}})]}{\sigma_s \sigma_{\hat{s}}}, \text{ if } \sigma_s \sigma_{\hat{s}} > 0 \quad (3.20)$$

where σ_s and $\sigma_{\hat{s}}$ are the standard deviations, μ_s and $\mu_{\hat{s}}$ the expected values and E the expected values operator [Ben+09]. The result of the Pearson coefficient can be interpreted as seen in Table 2, where negative values mean inverse association:

$\rho_{s,\hat{s}}$	$\rho_{s,\hat{s}}$	Association Between Variables
+0.8 to +1.0	-0.8 to -1.0	Very strong association
+0.6 to +0.8	-0.6 to -0.8	Strong association
+0.4 to +0.6	-0.4 to -0.6	Moderate association
+0.2 to +0.4	-0.2 to -0.4	Weak association
+0.0 to +0.2	-0.0 to -0.2	Very weak or no association

Table 2: Interpretation of the Pearson coefficient

The problem is that both input signals are assumed to be two random variables which is technically not the case. Although correlation has been used successfully in computational audio tasks such as simultaneous sound event localization [Cor+19] using a statistical relationship to compare a reference to a test signal proved challenging (see Section 4.2).

3.3.3 SI-SNR

The scale-invariant signal-to-noise ratio

$$\text{SI-SNR} = 10 \log_{10} \left(\frac{\|as\|^2}{\|as - \hat{s}\|^2} \right), \text{ where } a = \frac{\hat{s}^T s}{\|s\|^2} \quad (3.21)$$

measures the level of distortion or noise in the predicted signal in a way that is invariant to the scaling of the signals. It has been used successfully in dereverberation tasks [LM19] but while providing invariance to signal scaling it too does not conform to the perceived loudness of the human ear nor provide invariance to signal shifting.

It can also be mentioned that there are other variants, like Source-to-Artifact Ratio (SAR), Source-to-Interference Ratio (SIR), Source-to-Distortion Ratio (SDR) and Signal-to-Noise Ratio (SNR), each with Scale-Invariant (SI) forms, which are used in the field of source separation and speech enhancement, but are all inspired by the usual definition of the SNR [VGF06].

3.3.4 PESQ

Answering the shortcoming of metrics like the MSE and SI-SNR, the Perceptual Evaluation of Speech Quality (PESQ) model (a successor to the Bark Spectral Distortion (BSD) and Perceptual Speech Quality Measure (PSQM) models) is

both invariant to signal scaling and shifting. The PESQ score reflects speech quality on a continuous scale ranging from 1 to 5 (cf. Table 3)

Rating	Label
5	Excellent
4	Good
3	Fair
2	Poor
1	Bad

Table 3: The Absolute Category Rating scale used by mean opinion score (MOS)/PESQ

The scale shown in Table 3 corresponds to the MOS scale. During analysis the signal is mapped into a representation of perceived loudness in time and frequency through a psychoacoustic model based on the bark scale [Rix+01] which is a psychoacoustical scale on which equal distances correspond with perceptually equal distances [Zwi61] therefore assuring conformity with the human auditory system (cf. Figure 8).

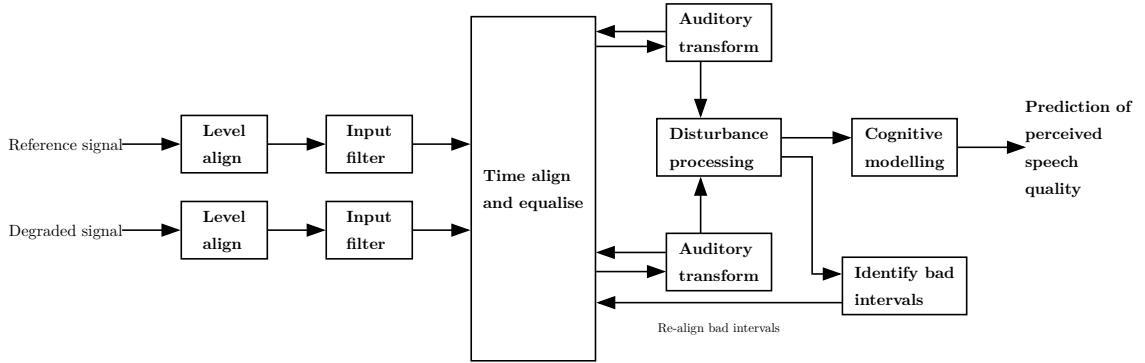


Figure 8: Structure of Perceptual Evaluation of Speech Quality (PESQ) model taken from Rix et al. (2001).

3.3.5 PEAQ

The Perceptual Evaluation of Audio Quality (PEAQ) model is based on the Perceptual Audio Quality Measure (PAQM) model and has been an ITU-R recommendation since 1999 [Rix+01]. In general it compares two time aligned signals, one processed and one original. Concurrent frames of each signal are transformed to a basilar membrane representation whose differences are further analyzed by

a cognitive model [Thi+00] (cf. Figure 9). The two offered metrics, namely the objective difference grade (ODG) and distortion index (DI), are therefore not invariant to signal shifting but they conform to the human perception of sound loudness. The objective difference grade (ODG) corresponds with the subjective difference grade (SDG) and indicates the audio quality of the tested signal on a continuous scale from -4 (very annoying impairment) to 0 (imperceptible impairment). The distortion index (DI) is a quality indicator like the ODG except for its higher sensitivity towards very low signal qualities [KAD17].

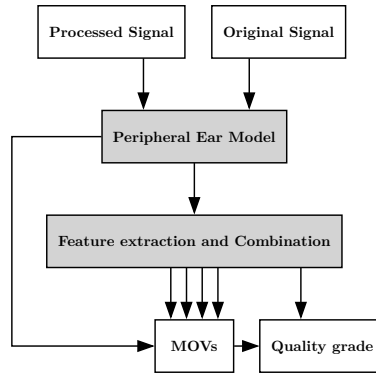


Figure 9: High-level representation of the Perceptual Evaluation of Audio Quality (PEAQ) model taken from Thiede et al. (2000).

4 Methodology

4.1 Dataset

Other machine learning fields mainly computer vision (CV) and large language model (LLM) have long been trained on publically available diverse datasets [Den+09] namely mC4, MassiveText or the Wikipedia dataset [Nav+25].

As shown in Chapter 2 previous work in the field of audio dereverberation has generally focused on speech signals. As this limitation is the same for many audio based machine learning problems (e.g. multi speaker separation, noise cancellation and speech to text) many of the most used large audio datasets, like LibriSpeech or WSJ0, consist only of speech signals which are reduced in bandwidth as well as language diversity and recorded in anechoic conditions [Gar+07, Pan+15, Ric+24].

Datasets of diverse audio signals have emerged from audio classification problems. Early examples being private self collected datasets of individual researchers [Ell01, Woo92]. Over the recent years interest in audio classification has surged as can be seen in the amount of entries in the Detection and Classification of Acoustic Scenes and Events (DCASE) challenge series that increased from 31 in 2013 to 428 in 2023 [Mes+24]. The DCASE has also been a major influence in the increase of publically available datasets as prior to the DCASE challenges only a limited amount were available most notably RWCP [Smi10].

The current largest dataset of diverse audio signals is Google’s fittingly named AudioSet containing over 5,800 hours of audio recordings with 527 classes of annotated sounds [Gem+17]. These recordings are 10 second clips drawn from YouTube videos. Building on top of the AudioSet classes the FSD50K dataset contains 100 hours of audio composed of 51,197 individual samples [Fon+22] taken from the “freesound.org” audio sharing site. The FSD50K dataset is publically available while AudioSet released embedding features of the raw audio data necessitating a private download from YouTube. Both datasets are human-labeled while AudioSet specifies that sounds are human-verified and classes are suggested using YouTube metadata.

4.1.1 Data Collection

Our proposed approach requires a diverse dataset of dry audio data. In total 108,775 individual audio samples were collected resulting in the following dataset:

Dataset	Number of Files	Length of Files
<i>AudioSet</i>	10790 (9.92 %)	44h 52m 34s (13.8 %)
<i>FSD50K</i>	46753 (42.98 %)	107h 34m 25s (33.1 %)
<i>LibriSpeech</i>	51232 (47.1 %)	172h 17m 49s (53.1 %)
<i>Total</i>	108775	324h 44m 49s

Table 4: Dataset composition

Diverse audio data from the AudioSet and FSD50K datasets were downloaded in 44.1 kHz. Both datasets were used as to eliminate any bias occurring in one of the datasets (e.g. YouTube compression artifacts). The LibriSpeech dataset [Pan+15] includes english utterances recored in anechoic conditions and sampled at 16 kHz. These were included in hopes of giving speech signals a greater weight as we felt clean speech was underrepresented in the other datasets.

A final dataset split of 70% training, 15% validation and 15% testing data was decided. Each sample was randomly assigned to one subset allowing for equal distribution of the entire dataset (cf. Table 4) in each subset.

To assure reproducibility the file name of each sample is hashed using MD5 [Riv92]:

$$\begin{aligned}
 h &= \text{MD5}(\text{name}) \\
 r &= \frac{\text{int}(h_{0\dots3})_{\text{LE}}}{2^{32} - 1}
 \end{aligned}
 \tag{4.1}$$

The first 4 bytes are then used to assign the split:

$$\text{split}(r) = \begin{cases} \text{train} & \text{if } r < S_{\text{train}} \\ \text{val} & \text{if } S_{\text{train}} \leq r < S_{\text{train}} + S_{\text{val}} \\ \text{test} & \text{otherwise} \end{cases}
 \tag{4.2}$$

With $S_{\text{train}} = 0.7$ and $S_{\text{val}} = 0.15$. A final distribution as can be seen in Table 5 was achieved.

Subset	Number of Files	Length of Files
<i>Train</i>	76234 (70.08 %)	227h 45m 06s (70.13 %)
<i>Validation</i>	16120 (14.82 %)	48h 16m 47s (14.87 %)
<i>Testing</i>	16421 (15.1 %)	48h 42m 54s (15.0 %)
<i>Total</i>	108775	324h 44m 49s

Table 5: Subset composition

Another dataset of room impulse responses (RIRs) was gathered, which was later in part used for reverberation purposes. The RIRs were collected from the Aachen Impulse Response (AIR) dataset [JSV09] as well as from the “Hybrid Reverb” plugin in Ableton Live 12¹. The AIR dataset contains 433 individual RIRs with different acoustical properties, such as reverberation time and room volume.

4.1.2 Dataset Creation

A supervised training approach (as explained in Section 3.2.2) was chosen to train both our dereverberation (cf. Figure 10) and perceptual loss model (cf. Figure 11). Labeling was done automatically through synthetic reverberation of the dry audio samples included in the dataset described in Section 4.1.1.

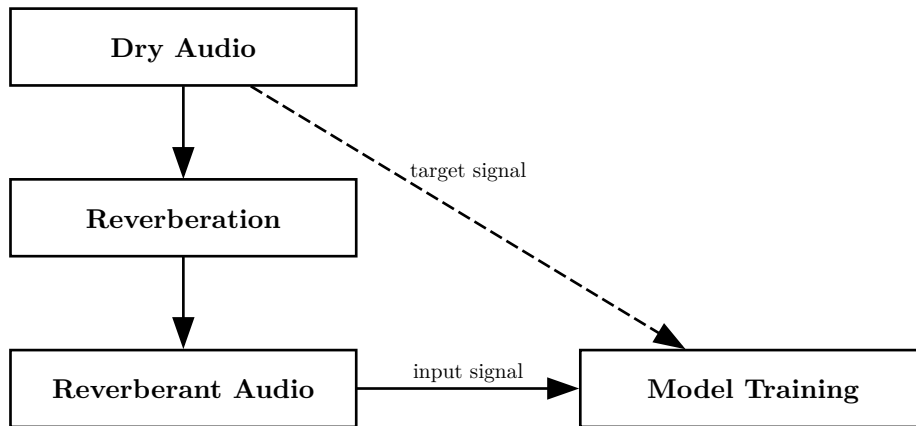


Figure 10: Dereverberation preprocessing pipeline

Additional labels used for the perceptual loss model (see Section 4.3) were saved during parameter based reverberation (see Section 4.1.2.1) and calculated from the dry-reverberant-sample pairs (see Section 4.1.2.2).

¹<https://www.ableton.com/en/packs/hybrid-reverb/>

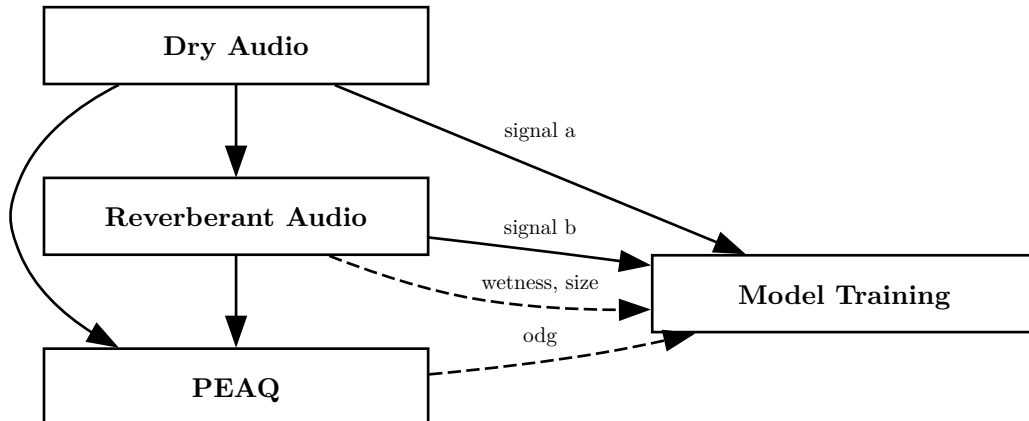


Figure 11: Perceptual loss preprocessing pipeline

This synthetic labeling approach is similar in concept to self-supervised training where a supervisory signal is generated through augmentation. For instance in computer vision tasks self-supervision is often used for autoencoder training or classification. Even in the domain of computational audio self-supervised approaches have shown great efficiency [Bae+20]. However as our objective is neither autoassociative nor contrastive but a supervised regression from reverberant to dry audio it cannot be classified as such (see Section 3.2.2).

4.1.2.1 Reverberation

To provide the model with reverberant audio signals two kinds of preprocessing approaches were considered. The signals could either be reverberated “*live*”, after loading a sample into memory during training, or “*offline*” beforehand saving compute time but sacrificing disk space.

The three main ways of digital reverberation are [Sch18]:

- convolutional
- delay networks
- computational acoustics

Reverberation through convolution via RIRs is the most realistic way of generating synthetic reverb, as it mimics the scattering characteristics of a real-world room at the RIRs recording position [Far07]. Generally this comes at a higher computational cost and latency [MNT16, Sid20]. Unfortunately convolution reverbs do

not expose many parameters or controls, making labeling of samples which are fed to our loss network (see Section 4.3) difficult.

Parameter based reverberation, like delay networks, is fast and requires little memory, but careful tuning is necessary to find configurations that sound realistic [Sch18, Sid20]. This gives us easy access to, e.g., size and wetness controls that we can use for labeling (see Section 4.3).

In computational acoustics room simulations are used for reverberating audio [Lem+23]. Game engines such as Unity [Man+25] or libraries like pyroomacoustics [SBD18] can be used to simulate rooms with different sizes, materials and microphone placements. This is done either by trying to solve the wave-equation by the discretization of the space, geometric solutions like the Image Source Method (ISM) [AB79] or ray tracing [Vor08]. While this is attempting to recreate an acoustic space as close as possible, it is also the most computationally expensive and not possible to do live or offline for our amount of data. Unity’s processing is also done in real time, which makes it not feasible, as the runtime would be about 324 hours (cf. Table 4).

A first implementation was done using live processing in memory. All three approaches described above were implemented for an interchangeable framework. Using this the original Conv-TasNet dataloader was adjusted to use the RIR implementation.

After getting access to the RWTH Aachen CLAIX compute cluster, we pivoted to an offline dataset as compute time was of more importance than disk space. As mentioned parameter based reverberation was suited best for our own networks, due to the access to size and wetness controls.

Specifically, we first used Valhalla Supermassive in VST3 format, which was later abandoned, as it lacked Linux compatibility. We then chose a FreeVerb implementation [Smi10] in the `pedalboard` Python package [Sob23].

When processing the data from our dataset, the decision was made to reverberate all files in their native sample rate and then later upsample them to the

sample rate used for training. The three sub-datasets have the following sample rates: AudioSet at 44.1 kHz, LibriSpeech at 16 kHz, and Freesound at 44.1 kHz.

While 44.1 kHz is a fairly standard sample rate for consumer audio content [Pu06], 16 kHz only allows for an upper frequency of

$$f_{\max} = \frac{f_s}{2} = \frac{16 \text{ kHz}}{2} = 8 \text{ kHz} \quad (4.3)$$

to be represented as shown by the Nyquist theorem [Sha49]. While this is technically enough to represent speech signals, which only need a bandwidth of 300 Hz to 3400 Hz [ITU88], we introduce some inconsistencies in the reverberation. The effects of these are discussed in Section 7.4.

4.1.2.2 Calculation of PEAQ Scores

For every dry-reverberant-sample pair the PEAQ scores ODG and DI (see Section 3.3.5) were calculated. As the GStreamer implementation “GstPEAQ” was used [HZ15], GStreamer Python bindings were utilized to automate this process [GSt26]. This approach meant we needed both reference and test files written to disk making a live implementation not feasible. All samples were upsampled to 48 kHz for use with PEAQ.

4.1.2.3 Non-Silent Parts

Using the Python package Pydub [Rob26] non-silent ranges of all samples of the training subset (see Section 4.1.1) were analyzed. A sample is considered silent if its level is below -40 dBFS. A range of samples is considered silent once it is longer than 100 samples.

Using this formula $d_{\text{full}} \approx 227.75$ hours of training data was examined. The duration of all non-silent ranges was $d_{\text{non_silent}} \approx 149.77$ hours leaving $d_{\text{silent}} \approx 77.98$ hours of silence or about

$$\frac{d_{\text{silent}}}{d_{\text{full}}} \approx 34.24\% \quad (4.4)$$

. The problem is worsened by the fact that samples shorter than the segment length defined in Section 5.5.1 are zero padded to the desired length adding even more silent parts.

As we don't want our model to focus on generating silence a mask is generated for each sample specifying its silent ranges (cf. Figure 12). This mask is then used in the loss function to ignore the silent range (see Section 5.5.2).

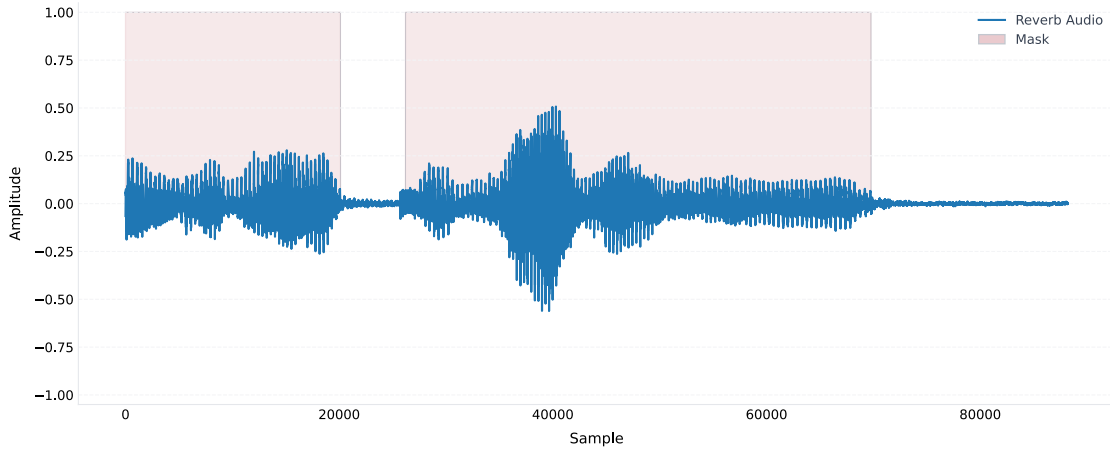


Figure 12: Signal with non-silent mask

4.2 Analyzation of Applicable Loss Functions

Jonathan Kron

As described in Section 3.2.3.1 a loss function is a qualitative function that is used to objectively measure model performance by calculating the deviation of the model's prediction to their ground truth counterpart. This deviation is mapped onto a real number that intuitively represents some error. To optimize model performance this error must be minimized.

As shown in Section 3.2.3.4 different loss functions exist for different problem sets. Each research endeavor in machine learning must decide which loss function to use based on the nature of the problem, the data available and the type of machine learning algorithm to be solved [Cia+24].

In the time or waveform domain error-based regressive loss functions (e.g. MSE, SI-SNR and PESQ) have identified themselves as well performing in the field of dereverberation (see Chapter 2 and Section 3.3).

The metrics described in Section 3.3 can all be used as loss functions. The problem is that all of them have in common that none are specific to our task of dereverberation. PESQ comes close to being a perceptual scale- and shift-invariant metric but as it is made for the evaluation of speech signals, effectiveness in diverse audio signals is doubtful. Rix et al. (2001) write: “Certain other applications have not yet been fully characterised or may need parts of the model to be changed. These include: music quality [...]”. An alternative lies in the Perceptual Evaluation of Audio Quality (PEAQ) model (cf. Section 3.3.5).

Other state-of-the-art measures include Google’s Virtual Speech Quality Objective Listener (ViSQOL) [Chi+20] as well as PEMO-Q [HK06] (cf. Section 3.3). But comparison indicates that PEAQ’s performance is not only competitive but sometimes even superior to newer approaches [DH20] meaning that ViSQOL and PEMO-Q were not further considered.

As explained in Section 4.1.2.1 the final dataset was reverberated offline using an implementation of the FreeVerb reverberator allowing for export of size and wetness parameters on a per sample basis. The wetness and size parameters are objective measurements which we know to be true. A fully reverberated signal is defined as:

$$(\text{wetness} = 1) \wedge (\text{size} = 1) \tag{4.5}$$

. A fully dereverberated signal is defined as:

$$(\text{wetness} = 1) \vee (\text{size} = 1) \tag{4.6}$$

. This enables us to plot the different quality metrics against these objective measures and assess their applicability for the dereverberation task. Or in other words how well each metric estimates reverberation (and in turn dereverberation) of a signal.

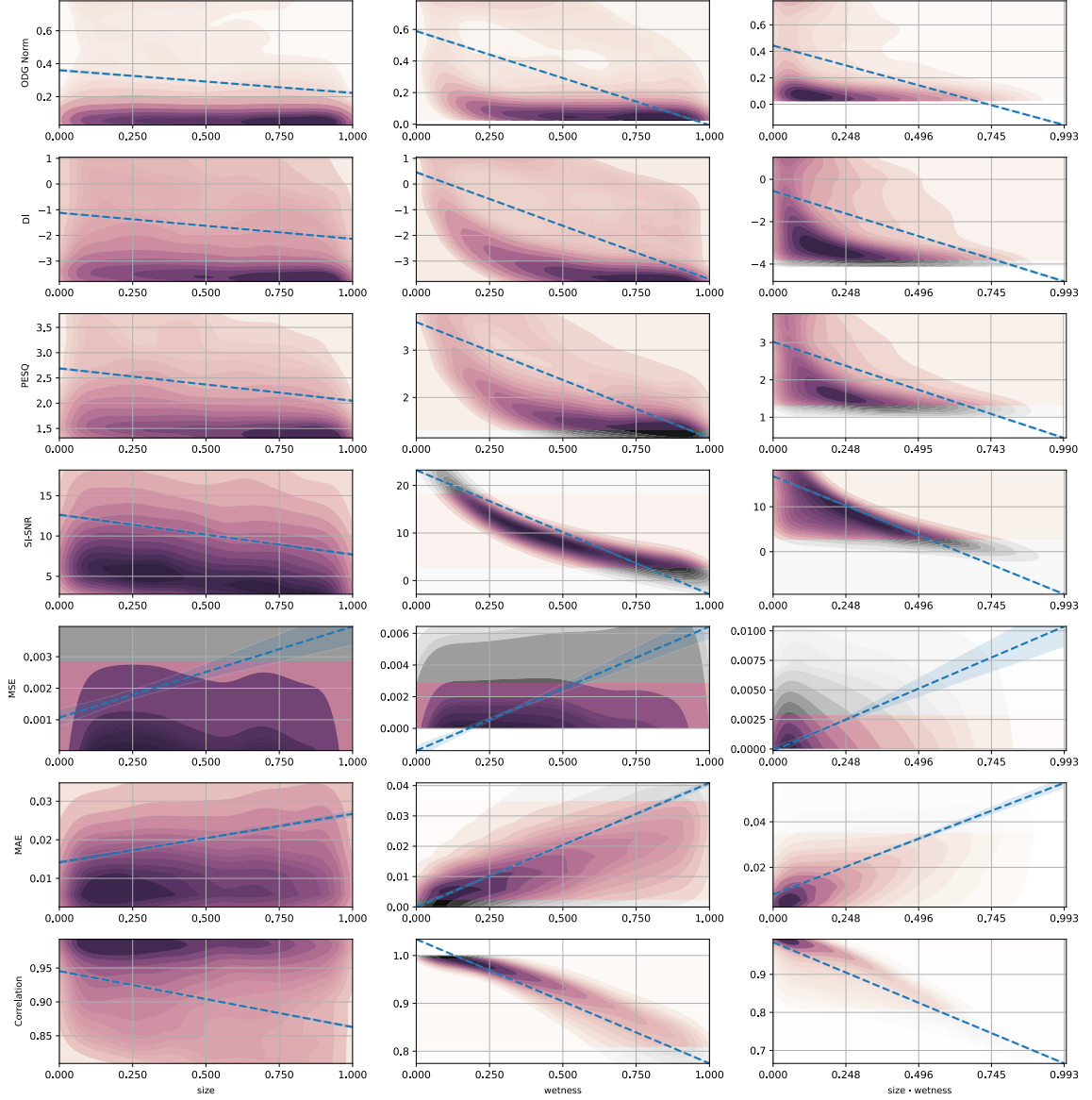


Figure 13: Metrics usable as loss functions analyzed over 16421 datapoints from test dataset (cf. Table 5), data between the 15th and 85th percentile is shown in color

Figure 13 shows a two dimensional kernel density estimation (KDE) for each metric plotted against both the size and wetness parameters as well as “size · wetness”. The latter one was included as it is possible that samples with high size values simultaneously exhibit low wetness values and therefore are not reverberant (cf. Equation (4.5) and Equation (4.6)). The “size · wetness” plot corrects for that.

A kernel density estimation (KDE) plot is similar to a histogram but differentiates itself through a continuous density curve. This density curve was then

subdivided into 15 distinct plateaus or levels where contour lines were drawn. All data shown in color lies between the 15th and 85th percentile of data points therefore excluding outliers. All data shown in grey is considered outlier data and is only displayed to fill space appropriated by the regression line.

The dotted blue line represents a linear regression over all data points including outliers. The light blue confidence interval band represents the 95% confidence interval for the regression line.

All tested signals were time and amplitude aligned. The only difference being the reverberation of the processed signal. Wetness and size values of the reverberator are selected randomly from a uniform distribution.

Prior to analysis the ODG score was normalized

$$\text{ODG}_{\text{norm}} = \left(\frac{\text{ODG} + 4}{4} \right)^{\perp_1}_{\tau_0} \quad (4.7)$$

. This procedure did neither aid nor hinder analysis but meant that the interpretation of the absolute value of the ODG score changes from “lower is better” to “lower is worse”. It was carried out as a remnant from early tests described in Section 4.3. Figure 13 shows that ODG is not a good indicator of dereverberation performance as most values stay between 0 and 0.2 in similar densities for the entire range of size and wetness values meaning that most test signals even those with little reverberation were classified as annoyingly impaired.

The DI score was not normalized as exact value range is unknown to us. It behaved similarly to the ODG score but showed slightly better performance against the wetness parameter but arguably worse performance against the size parameter where many strongly reverberated signals are classified as “good”.

Although PESQ was only proven to work on speech signals it showed a slightly improved performance compared to the DI score.

It is evident that the SI-SNR metric performs best as a judgement of dereverberation performance. The wetness KDE plot shows a strong correlation of absolute SI-SNR value and reverberation influence. And as wetness and size values are

randomly sampled from a uniform distribution the SI-SNR density stays mostly the same over the entire wetness range which is the desired behavior.

Although more outlier data is present in the size plot a clear downward trend can be examined in the highest density parts of the KDE plot. Further strengthening the assessment that SI-SNR predicts reverberation well in diverse audio signals.

The MSE metric shows not only no real predictive performance in the KDE plot but also a broad confidence interval negating the expression of the regression line.

In similar fashion, the MAE shows poor performance against the size parameter. The wetness plot shows a slight increase of MAE against increasing wetness values. The problem being that this increase happens all below 0.04 meaning that every wetness value is assigned a very low error value or in other words is interpreted as “good”.

The correlation metric exhibits a similar problem where not only the size plot shows subpar performance but the values in the wetness plot range from 1 to 0.8 which signifies a “very strong association” across the entire wetness range (cf. Section 3.3.2) making interpretation of the correlation value with respect to reverberation challenging. Furthermore the density of the correlation-wetness plot does not align with the uniform distribution.

4.3 Perceptual Quality Network

Jonathan Kron

Although Section 4.2 shows the SI-SNR metric to have good qualities regarding the assessment of dereverberation performance in diverse audio signals according to the wetness parameter, the size parameter is not well represented. Calculating exact truths about a reverberated signal without the use of a neural network is near impossible, as it either requires knowledge of the sound source or the ability to model the reverb tail which is not possible in short continuous utterances [Rat+03]. The SI-SNR like all metrics introduced in Section 3.3 suffers from the need of a “golden” reference which as Fu et al. (2018) write “considerably restricts

the practicality of such assessment tools [...]”. The presence of MOS tests shows that humans can evaluate signal quality without the need of such a reference signal [Fu+18]. Motivated by these shortcomings we introduce our own loss network initially coined “Perceptual Quality Network”.

We place the following requirements on this loss network. It must be differentiable as it is to be used as a loss function (see Section 3.2.3.1). As we plan to use it on a dataset of diverse audio signals (cf. Section 4.1.1) it must support wideband analysis up to 44.1 kHz.

Differentiability is given by the fact that the computations of a neural network are differentiable. The only exception being the activation function ReLU which does not have a derivative in $z = 0$. But in a real application the gradients are almost never zero so this was ignored.

Similar in nature to Quality-Net [Fu+18] which estimates a PESQ score for a given signal (see Section 2.4) our initial idea was to estimate a combination of the ODG score, which we then thought best, as well as size and wetness parameters. Therefore our model combines a perceptual and an objective approach.

Every training example combines a reverberated audio sample as the input for the neural network, the normalized size and wetness parameters used for the reverberation of the input audio sample, the normalized ODG score taken from the comparison to the original sample using PEAQ and a quality score defined as:

$$\text{quality} = \text{ODG}_{\text{norm}} \cdot (1 - \text{wetness}_{\text{norm}} \cdot 0.4) \cdot (1 - \text{size}_{\text{norm}} \cdot 0.3) \quad (4.8)$$

. This quality score is used as a loss in the dereverberation network (see Section 5.4).

Implementation details regarding model architecture and loss functions used qualify ODG, size, wetness and quality predictions are discussed in Section 5.2. Results are shown in Section 6.3 and an evaluation of the performance of the perceptual quality net is found in Section 7.1.

4.4 Objective Quality Network

Section 4.2 shows PEAQ as being a flawed metric to estimate dereverberation performance. This conclusion is supported by the findings in Section 7.1. Challenging the idea that a perceptual metric is best we decided to test a purely objective network as a loss function.

Adopting the same structure as the perceptual quality network the objective quality network only estimates the size and wetness attributes by defining the quality score as:

$$\text{quality} = 1 - 0.6 \cdot \text{wetness} - 0.4 \cdot \text{size} \quad (4.9)$$

Wetness was given more importance because Section 6.3 has shown the network as having better estimation performance for the wetness parameter.

Implementation details regarding model architecture and loss functions are discussed in Section 5.3. Results are shown in Section 6.3 and an evaluation of the performance is found in Section 7.2.

5 Implementation & Experimental Setup

5.1 Conv-TasNet for diverse audio dereverberation Leo Kling

Conv-TasNet [LM19] operates in the time domain using a learned encoder–TCN–decoder architecture, where a temporal convolutional network estimates a multiplicative mask over the encoded signal to isolate a target source (cf. Section 2.1). Although originally designed for speech source separation, the masking paradigm is conceptually compatible with dereverberation. Late reflections overlap with the direct sound in the encoder representation, and a mask can in principle suppress this reverberant energy while retaining the direct component.

No pre-trained dereverberation weights were publicly available. Weights linked from the original repository were trained for speaker separation only and are thus not applicable to this task. Attempts to obtain suitable weights from the original authors received no reply. We therefore trained the model from scratch using the implementation linked in the paper².

The encoder is a 1-D convolution (512 channels, window 2 ms at 8 kHz). The TCN separator consists of 8 layers across 3 stacks with feature dimension 128 and depthwise-separable convolutions of kernel size 3. The decoder mirrors the encoder via a transposed convolution. The model is configured as a single-source system. The 8 kHz sample rate imposes a hard frequency ceiling of 4 kHz, which excludes upper harmonics and air that are perceptually important for music and broadband diverse audio content.

The original training sets WSJ0-2mix and WSJ0-3mix [Gar+07] used in the Conv-TasNet paper are not publicly available, requiring an alternative training dataset. We used LibriSpeech [Pan+15], resampled to 8 kHz and segmented into random 4-second crops per iteration. Reverberation is applied on-the-fly by convolving the dry signal with one of five impulse responses from the preprocessing pipeline, yielding time-aligned wet/dry pairs with varied room conditions across epochs. The training dataset is therefore speech-only, which biases the mask priors

²<https://github.com/naplab/Conv-TasNet>

toward speech characteristics and is expected to reduce generalization to music and other diverse audio content.

Training used the Adam optimizer [KB17] with a learning rate of 10^{-3} , gradient clipping with maximum L_2 -norm of 5.0 [LM19], and a batch size of 32 over 100 epochs via PyTorch Lightning.

5.2 Perceptual Quality Network

Jonathan Kron

The perceptual quality network was implemented twice. Section 5.2.1 shows the initial implementation of the perceptual quality network. It features a simple encoder network and prediction heads for each scoring metric.

A second implementation based on CNN14 as introduced by Kong et al. (2020) was written to address the shortcomings of the first implementation as mentioned in Section 7.1.1.

5.2.1 Simple CNN

The initial implementation of the perceptual quality network is based on a simple two dimensional CNN. This architecture was chosen because CNNs have been widely adopted in audio machine learning [Gra+25] and have shown great performance. The small computational cost increase as compared to a waveform-domain approach was a nonissue as this network was not to be used during inference but only during training of the dereverberation model (see Section 5.4).

The forward pass includes conversion into a log-magnitude spectrogram using the STFT, logarithmic compression is discussed in Section 5.2.2, a shared encoder counting three two-dimensional convolutional layers all featuring batch normalization, ReLU as the activation function and Max Pooling.

The output of this shared encoder is then fed into three prediction heads each corresponding to one of the three initial labels (wetness, size, ODG). The output of each prediction head is concatenated with the shared encoder output and fed into the quality prediction head giving this model the ability to predict all parameters at once.

This gives us a better chance at debugging (cf. Section 7.1.2) and the opportunity to use just one of the four predictions as a loss function. In the end only the quality prediction was utilized.

Perceptual Quality Network (Initial)		
Log-magnitude spectrogram STFT: n_fft=2048, hop_length=512		
$\left(\begin{array}{c} 7 \times 7@32 \\ \text{BN, ReLU} \end{array} \right)$		
MaxPool 2×2		
$\left(\begin{array}{c} 5 \times 5@64 \\ \text{BN, ReLU} \end{array} \right)$		
MaxPool 2×2		
$\left(\begin{array}{c} 3 \times 3@128 \\ \text{BN, ReLU} \end{array} \right)$		
AdaptiveAvgPool (4×4)		
Flatten FC 2048 $\rightarrow$ 256, ReLU, Dropout(0.3)		
ODG Head FC 256 $\rightarrow$ 64 ReLU FC 64 $\rightarrow$ 1 Sigmoid	Size Head FC 256 $\rightarrow$ 32 ReLU FC 32 $\rightarrow$ 1 Sigmoid	Wetness Head FC 256 $\rightarrow$ 32 ReLU FC 32 $\rightarrow$ 1 Sigmoid
Concat [features(256) odg(1) size(1) wetness(1)] $\rightarrow$ 259		
Quality Head FC 259 $\rightarrow$ 64, ReLU FC 64 $\rightarrow$ 1, Sigmoid		

Table 6: Architecture of the initial implementation of the perceptual quality network

Table 6 shows the architecture of the initial implementation. The number after the “@” symbol indicates the number of feature maps. AdamW was used as an optimizer [LH19] with a learning rate of 10^{-3} . A per-prediction-head loss was calculated head using MSE. The total loss was defined as:

$$\text{loss} = 2 \cdot \text{loss}_{\text{quality}} + \text{loss}_{\text{odg}} + 0.75 \cdot \text{loss}_{\text{size}} + 0.75 \cdot \text{loss}_{\text{wetness}} \quad (5.1)$$

5.2.2 CNN14

Compared to the initial implementation this version offers a number of improvements. Mainly a new shared encoder architecture based on the CNN14 network (cf. Table 7) which was introduced as a real-time audio pattern recognition model by Kong et al. (2020). We thought it fitting as we were trying to solve an adjacent problem (audio characteristic recognition) with good performance as we did not want to needlessly slow down the training process of the dereverberation network.

A second improvement has been made in the spectrogram conversion. The log-magnitude spectrogram was replaced with a log-mel spectrogram offering a perceptually oriented base for the encoder. The mel scale compresses higher frequencies more than lower ones. Therefore mimicing human perception of audio. The conversion from hertz into mels is defined as [O'S87]

$$m = 2595 \cdot \log_{10} \left(1 + \frac{f}{700} \right) \quad (5.2)$$

. The mel scale was chosen in favor of the bark scale as it is the most used and best-performing scale in computational acoustics (e.g. in ASR [Dho+19, Sim26]). Prediction heads as well as loss calculations were not subject to change.

The logarithmic compression of the mel scale (similar to the compression in Section 5.2.1) is motivated by the human perception of loudness. Logarithmic compression also helps in the representation of the mel value range for use in neural network training. Without any compression the mel scale is condensed in a small range with extreme outliers. This means the network must be trained with higher numerical precision hence making it more susceptible to noise. It was shown that decibel-scaled melspectrograms always outperform the linear versions [Cho+17].

Weights of the shared encoder are initialized using the Xavier uniform distribution as described in [GB10].

Perceptual Quality Network (CNN14)		
Log-mel spectrogram sr=44100, n_fft=2048, hop=512, n_mels=128		
BN (128 mel bins)		
$\left(\begin{smallmatrix} 3 \times 3@64 \\ \text{BN, ReLU} \end{smallmatrix} \right) \times 2$		
AvgPool 2×2 , Dropout(0.2)		
$\left(\begin{smallmatrix} 3 \times 3@128 \\ \text{BN, ReLU} \end{smallmatrix} \right) \times 2$		
AvgPool 2×2 , Dropout(0.2)		
$\left(\begin{smallmatrix} 3 \times 3@256 \\ \text{BN, ReLU} \end{smallmatrix} \right) \times 2$		
AvgPool 2×2 , Dropout(0.2)		
$\left(\begin{smallmatrix} 3 \times 3@512 \\ \text{BN, ReLU} \end{smallmatrix} \right) \times 2$		
AvgPool 2×2 , Dropout(0.2)		
$\left(\begin{smallmatrix} 3 \times 3@1024 \\ \text{BN, ReLU} \end{smallmatrix} \right) \times 2$		
AvgPool 2×2 , Dropout(0.2)		
Global pooling (mean over freq.) Max + Mean pool over time $\rightarrow$ 1024-dim Dropout(0.5)		
FC 1024 $\rightarrow$ 512, ReLU, Dropout(0.3)		
ODG Head FC 512 $\rightarrow$ 128 ReLU, Dropout(0.3) FC 128 $\rightarrow$ 1 Sigmoid	Size Head FC 512 $\rightarrow$ 64 ReLU, Dropout(0.3) FC 64 $\rightarrow$ 1 Sigmoid	Wetness Head FC 512 $\rightarrow$ 64 ReLU, Dropout(0.3) FC 64 $\rightarrow$ 1 Sigmoid
Concat [features(512) odg(1) size(1) wetness(1)] $\rightarrow$ 515		
Quality Head FC 515 $\rightarrow$ 128, ReLU, Dropout(0.3) FC 128 $\rightarrow$ 1, Sigmoid		

Table 7: Architecture of the CNN14 based implementation of the perceptual quality network

5.3 Objective Quality Network

ODG was shown to worsen the performance of the perceptual quality network (see Section 7.1.2). Consequently the perceptual quality network as seen in Section 5.2.2 was adjusted to base the prediction only on the wetness and size parameters.

This change was done in two stages. Firstly only the quality score was modified by removing the ODG score (cf. Section 4.4). Loss calculation was left unchanged.

The second stage also changed the loss calculation from Equation (5.1) to just

$$\text{loss} = \text{loss}_{\text{quality}} \quad (5.3)$$

Both stages are evaluated in Section 7.2.

5.4 Dereverberation Network

Leo Kling

The model operates directly on waveforms. A reverberant input signal $\mathbf{x} \in \mathbb{R}^L$ is first projected into a learned latent space by a strided 1-D convolution $\mathbf{W}_{\text{enc}}$, processed by a TCN that predicts a real-valued mask, and finally reconstructed by a transposed convolution $\mathbf{W}_{\text{dec}}$:

$$\tilde{\mathbf{x}} = \mathbf{W}_{\text{dec}}^{\top} * (\sigma(\text{TCN}(\mathbf{W}_{\text{enc}} * \mathbf{x})) \odot (\mathbf{W}_{\text{enc}} * \mathbf{x})) \quad (5.4)$$

The sigmoid-gated mask $\sigma(\dots) \in (0, 1)^{C \times T}$ suppresses reverberation in the encoder feature space before the decoder maps the filtered representation back to a waveform estimate $\tilde{\mathbf{x}}$.

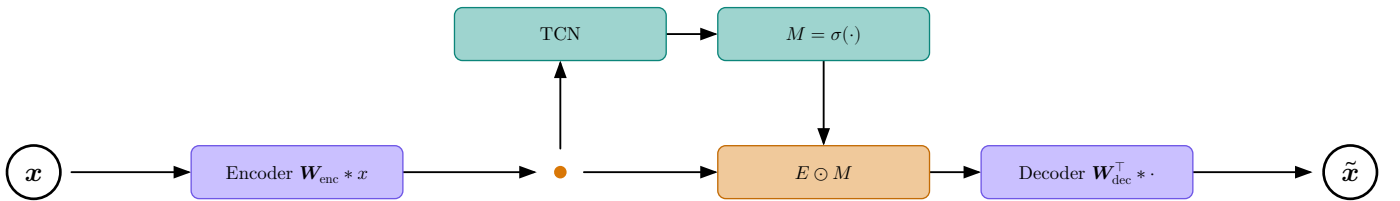


Figure 14: Forward pass of the dereverberation model.

- which versions do they want to show the implementation of? only the best one (then compare si-snr with perceptual?)

- show the architecture of the best version (v4)
- architecture is mostly the same for all
- show table with all hyperparameters (learning rate, batch size, etc.) for all versions

Use as loss function

eval() mode -> backpropagation not stopped through network computations -> what is frozen and what not

$$\text{loss} = 1.0 - \text{quality} \quad (5.5)$$

$$\text{loss} = 1.0 - \text{quality} + \alpha \cdot \text{MSE}_{\text{loss}} \quad (5.6)$$

inverse estimation in encoder space

- frequency
- time (ConvTasNet)
 - use conv tasnet mask
 - test for diverse audio signals
 - compare mse to perceptual loss

<https://typst.app/universe/package/neural-netz>

5.5 Placeholders

5.5.1 SOMETHING WITH SEGMENT LENGTH

5.5.2 LOSS FUNCTION SILENT MASK

6 Results

6.1 Conv-TasNet

Leo Kling

As explained in Section 5.1, we trained a Conv-TasNet model from scratch on the LibriSpeech `train-clean-100` split, using the original Conv-TasNet architecture [LM19] and training procedure, but with MSE instead of SI-SNR as the loss function.

Three loss functions were evaluated in total. The SI-SNR, which serves as the original Conv-TasNet training objective, did not converge: the loss remained negative throughout and continued to decrease without producing usable predictions, with the best checkpoint reaching a validation SI-SNR of -69.72 dB after 118 epochs (cf. Figure 15). A multi-scale spectral loss (MSS) likewise showed convergence but only at an unreasonably high value, reaching a validation loss of 165,861.84 after 119 epochs (cf. Figure 16). This could be attributed to some configuration choices that were set implicitly and thus often fail to provide informative gradients as claimed by Schwär and Müller (2023).

Switching to a standard MSE loss resolved the issue: training converged stably to a validation loss of approximately 0.0009 after 125 epochs (cf. Figure 15).

we do not know the reason for this, might be a user error

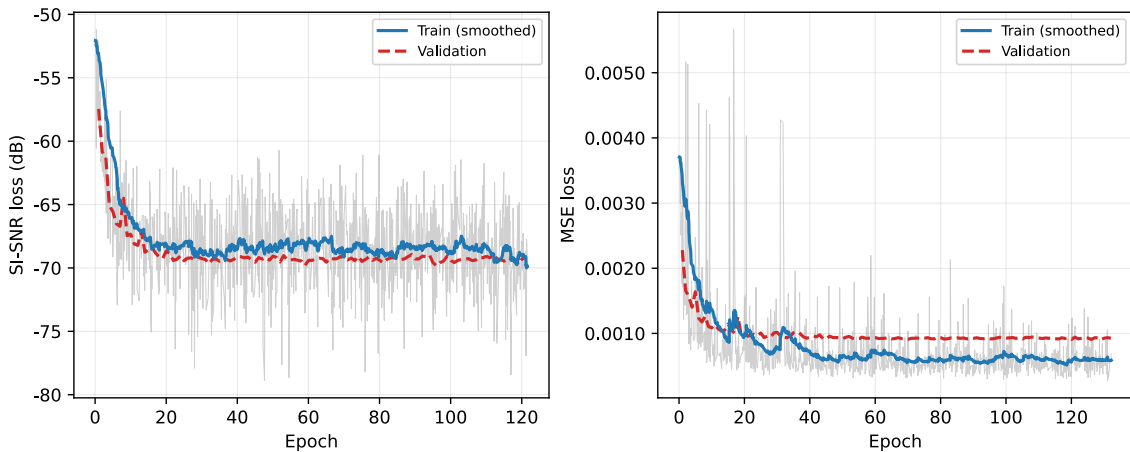


Figure 15: Training curves for Conv-TasNet with different loss functions. Smoothed using an exponential moving average with $\alpha = 0.05$.

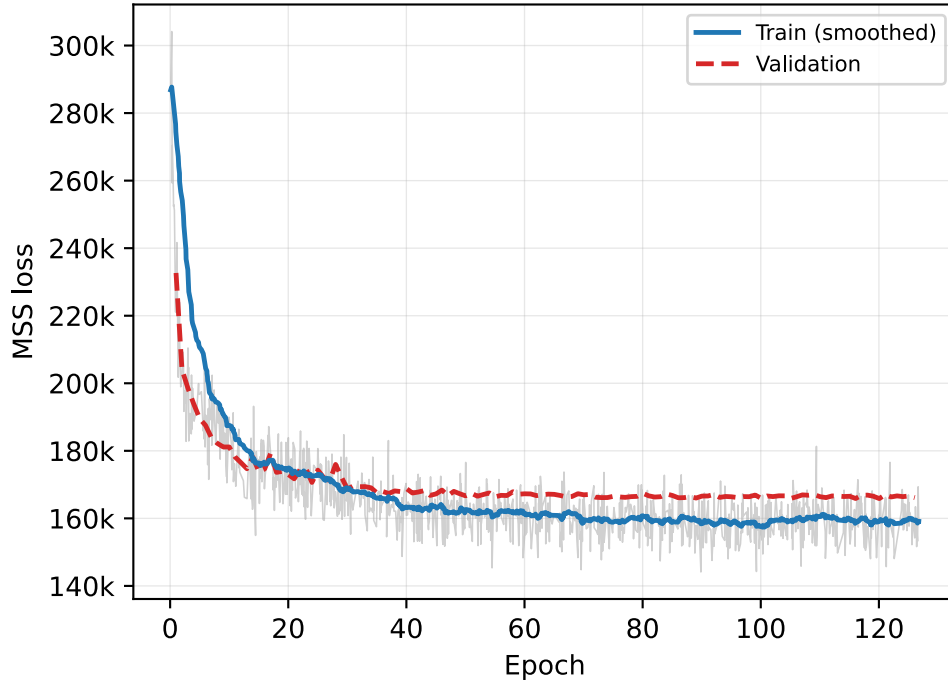


Figure 16: Training curve for Conv-TasNet with MSS loss. Smoothed using an exponential moving average with $\alpha = 0.05$.

The MSE-trained model was evaluated on the LibriSpeech **test-clean** split as well as on a diverse random subset taken from AudioSet covering speech, music, vehicles, and environmental sounds.

On speech samples the model reduces reverberation tails and produces audible dereverberation. It can also be observed that the model applies a low-pass filter, reducing high frequencies above about 2.5 kHz by about 20 dB (see Figure 17).

Detailed results of the evaluation done on the MSE-trained model can be seen in Table 8. A positive SI-SNR and scale-invariant signal-to-distortion ratio (SI-SDR) indicate dereverberation, while the PESQ score still indicates a “*poor*” performance (cf. Section 3.3.4)

write about Table 8, and how it may show signs of dereverberation (or it doesn't, but it's purely audible)

explain that the MSE checkpoint does return comparable SISNR values from the librispeech dataset as the original ConvTasNet paper reports

15.3 dB in [LM19] vs 10.0 dB in Table 8

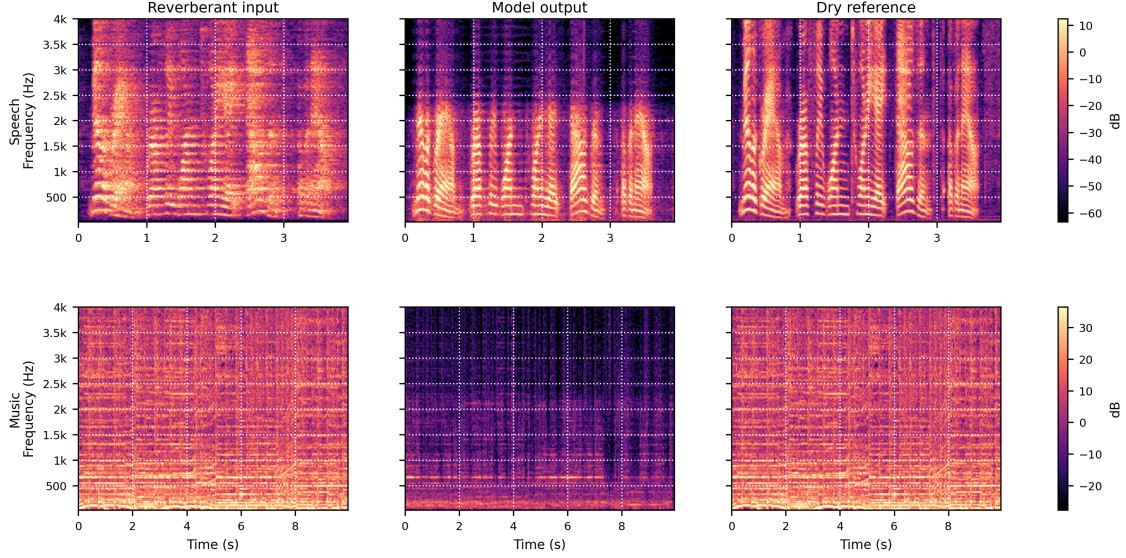


Figure 17: Spectrogram comparison of input (left) and output (middle) of the MSE-trained Conv-TasNet on a speech (top) and music (bottom) sample.

On music and non-speech content, however, the model introduces noticeable timbral artifacts.

Metric	Mean	Std	Min	Max
SI-SNR	10.036	3.856	−6.669	19.226
SI-SDR	10.003	3.872	−10.015	19.220
PESQ	1.726	0.304	1.117	3.015
WV-MOS	1.695	0.441	1.229	3.350

Table 8: Conv-TasNet dereverberation metrics on LibriSpeech **test-clean** ($N = 2620$)

These limitations – the 4 kHz bandwidth ceiling, speech-only training data, and the uncertainty of SI-SNR as a training objective for diverse audio – motivate the development of a dedicated dereverberation model trained on broadband diverse content and supported by a perceptual loss network.

6.2 StoRM

Leo Kling

Unlike Conv-TasNet, StoRM was not trained from scratch. We used the official pretrained dereverberation checkpoint provided by the authors, trained on the WSJ0 corpus reverberated with the REVERB challenge dataset [Gar+07, Kin+13, Lem+23]. The training data consists of speech recordings sampled at 16 kHz, establishing an 8 kHz frequency ceiling. Architecturally, StoRM follows a generative stochastic regeneration approach: a discriminative denoiser first produces an initial estimate of the clean signal, which a score-based diffusion model then refines through a learned reverse process [Lem+23]. This contrasts with Conv-TasNet’s discriminative masking, and the iterative inference required by the diffusion component has direct implications for computational cost.

On in-domain speech signals, StoRM achieves strong dereverberation quality. Table 9 shows the evaluation from the original paper. These numbers serve as an upper bound for speech dereverberation quality achievable with this model.

Method	WV-MOS	PESQ	ESTOI	SI-SDR	SI-SIR	SI-SAR
Mixture	1.78 ± 0.99	1.36 ± 0.19	0.46 ± 0.12	-7.3 ± 5.5	-7.5 ± 5.4	—
SGMSE+	3.49 ± 0.39	2.66 ± 0.45	0.85 ± 0.06	2.4 ± 7.2	11.6 ± 9.9	2.8 ± 6.8
NCSN++	2.99 ± 0.38	2.08 ± 0.47	0.85 ± 0.06	6.1 ± 3.8	21.4 ± 7.0	6.1 ± 3.7
GaGNet	2.40 ± 0.52	1.59 ± 0.37	0.68 ± 0.09	-0.5 ± 4.8	7.7 ± 4.0	0.2 ± 5.1
StoRM	3.73 ± 0.32	2.83 ± 0.42	0.88 ± 0.04	6.5 ± 4.0	22.9 ± 8.2	6.5 ± 3.9

Table 9: StoRM dereverberation metrics on the WSJ0+REVERB test set, reproduced from Lemercier et al. (2023).

In our own listening tests on speech samples, this quality is confirmed: reverberation tails are cleanly removed with rarely any audible artifacts (see Figure 18). Informally, the model appears to perform slightly worse on female voices, which may be attributable to a gender bias in the WSJ0 training corpus toward male utterances, even though the authors claim: “[...] about half the speakers are male and half female” [Gar+07]. Compared to Conv-TasNet, StoRM produces a markedly wider frequency response up to 8 kHz, avoiding the strong low-pass filtering effect observed in the MSE-trained Conv-TasNet output.

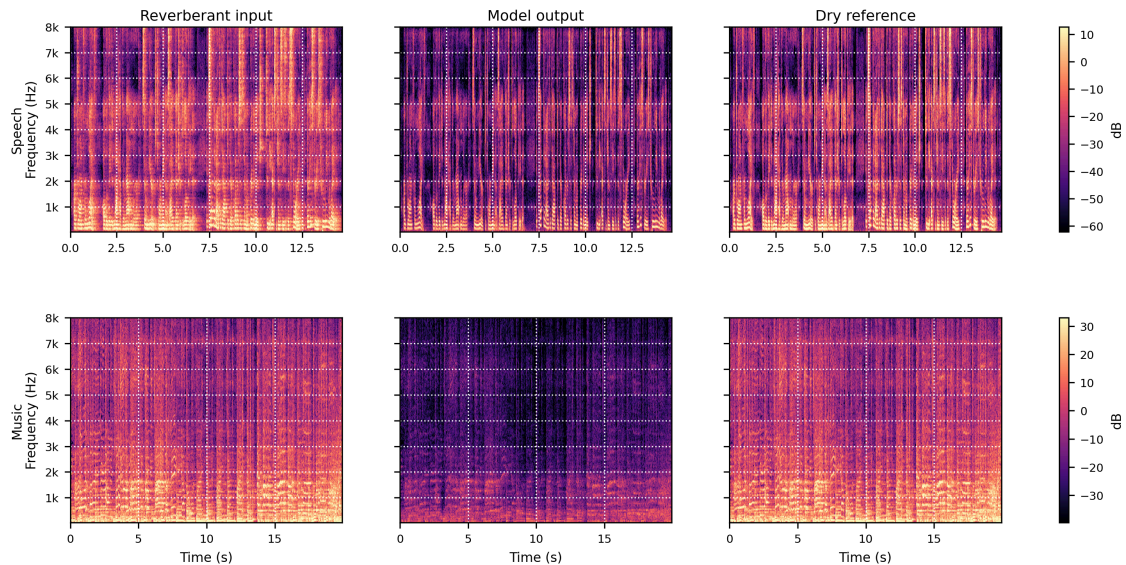


Figure 18: Spectrogram comparison of input (left) and output (middle) of StoRM on a speech (top) and music (bottom) sample.

On music and other non-speech content, the model’s behaviour is less predictable. As the pretrained checkpoint has no exposure to non-speech signals during training, generalisation is limited to the extent that spectral patterns of broadband audio are covered by the speech-domain prior. In listening tests, music samples processed by StoRM tend to exhibit subtle timbral changes compared to the unprocessed input, without achieving a consistent reduction of the reverberant tail. This out-of-domain degradation is expected given the training data composition and is explored further in the quantitative comparison in the next section.

The iterative reverse diffusion inference requires many sequential neural network evaluations per sample, making StoRM substantially more expensive than Conv-TasNet. On a single H100 GPU (CLAIX-2023-ML), processing 2048 AudioSet samples took 6 h 14 m 51 s, compared to 4 m 15 s for Conv-TasNet — approximately $88 \times$ slower (cf. Table 12). Real-time application of this pretrained model is therefore not feasible without architectural modifications such as reducing the number of reverse diffusion steps or distillation.

6.3 Perceptual Quality Network

Jonathan Kron

- all results based on the dataset in Section 4.1.1

- no results from initial perceptual quality network only cnn14 is discussed as
 - as Section 7.1.1 shows a direct improvement of cnn14 over initial implementation

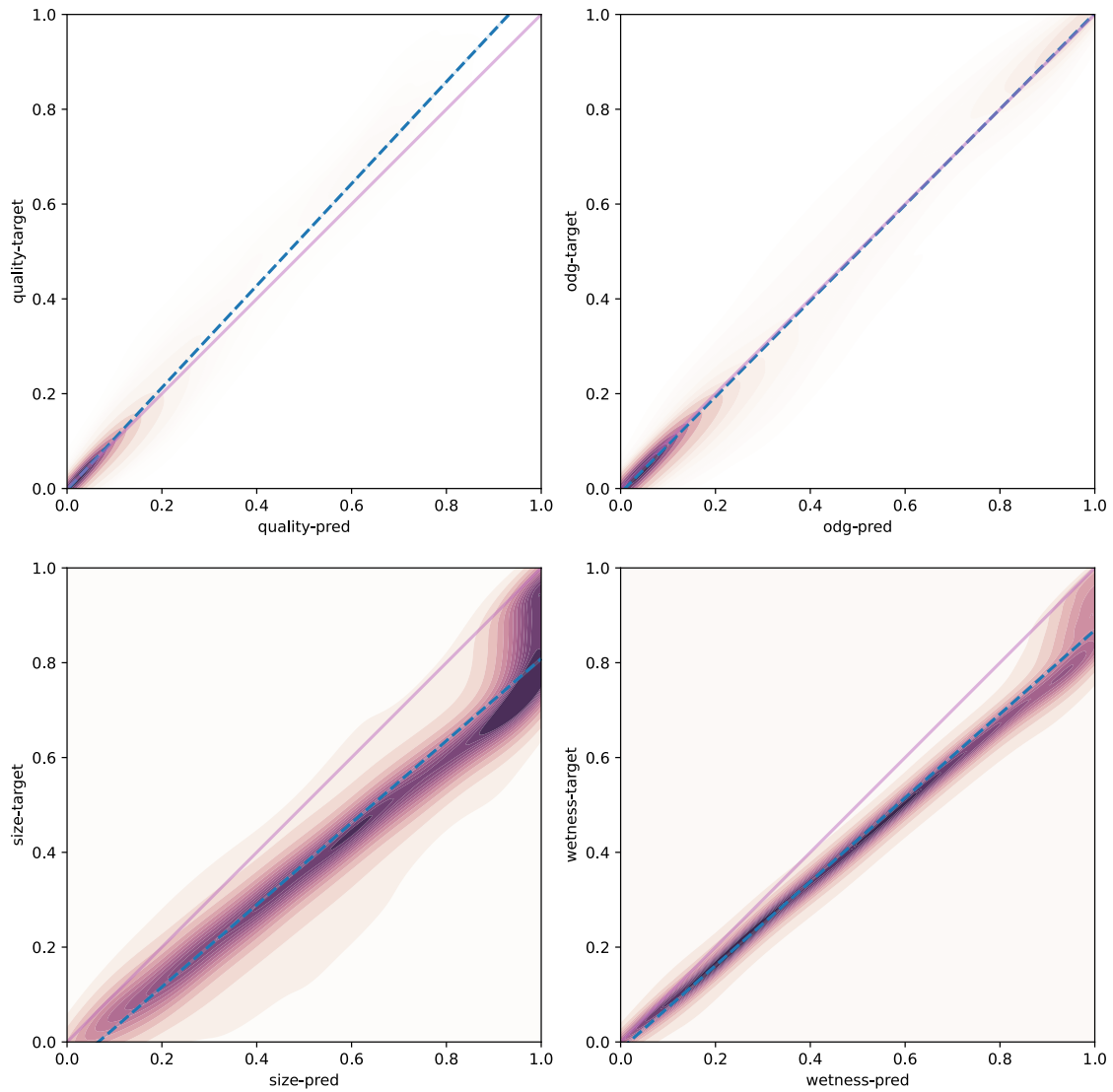


Figure 19:

6.4 Objective Quality Network

Jonathan Kron

update score:

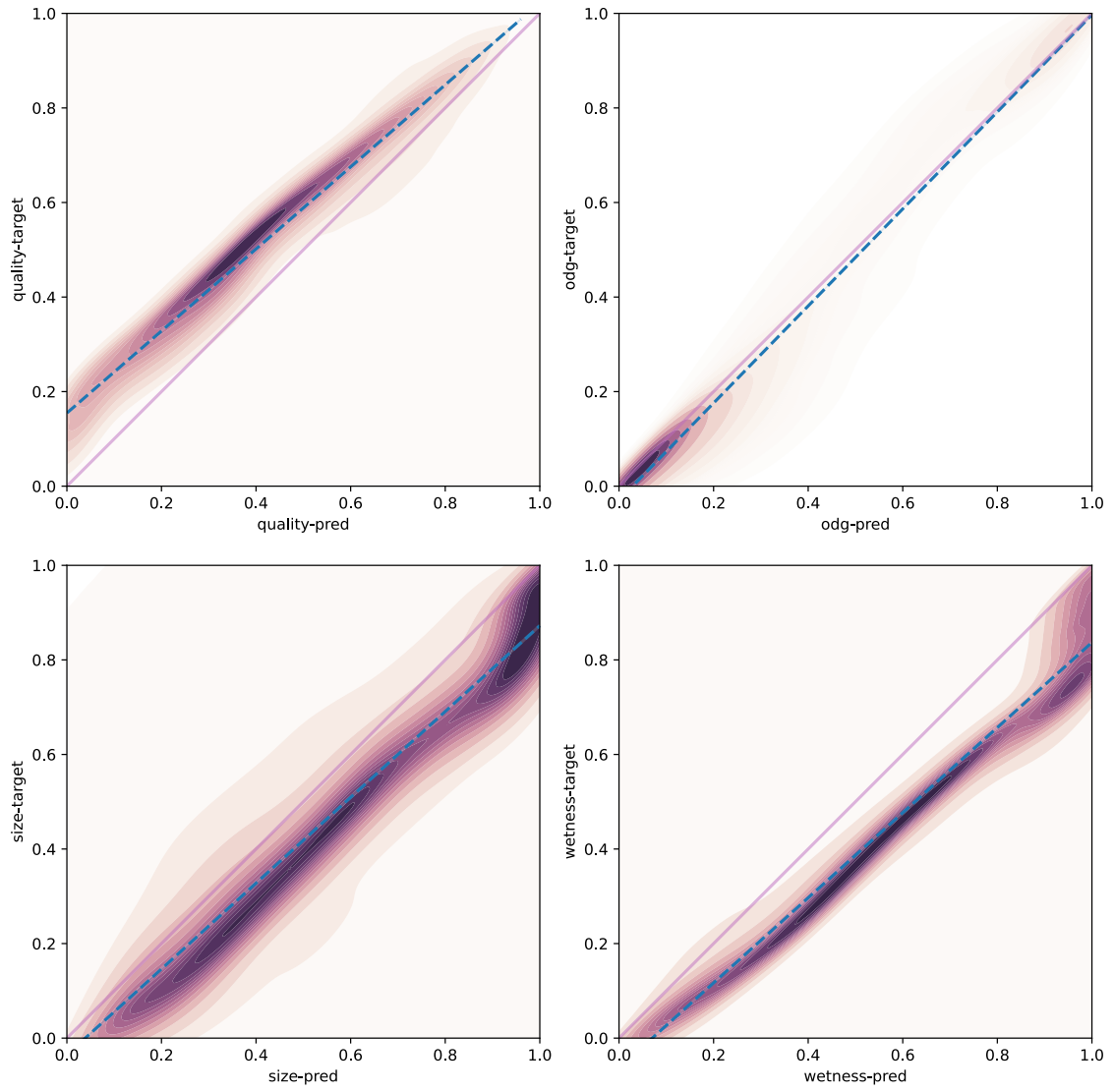


Figure 20:

update score and update loss:

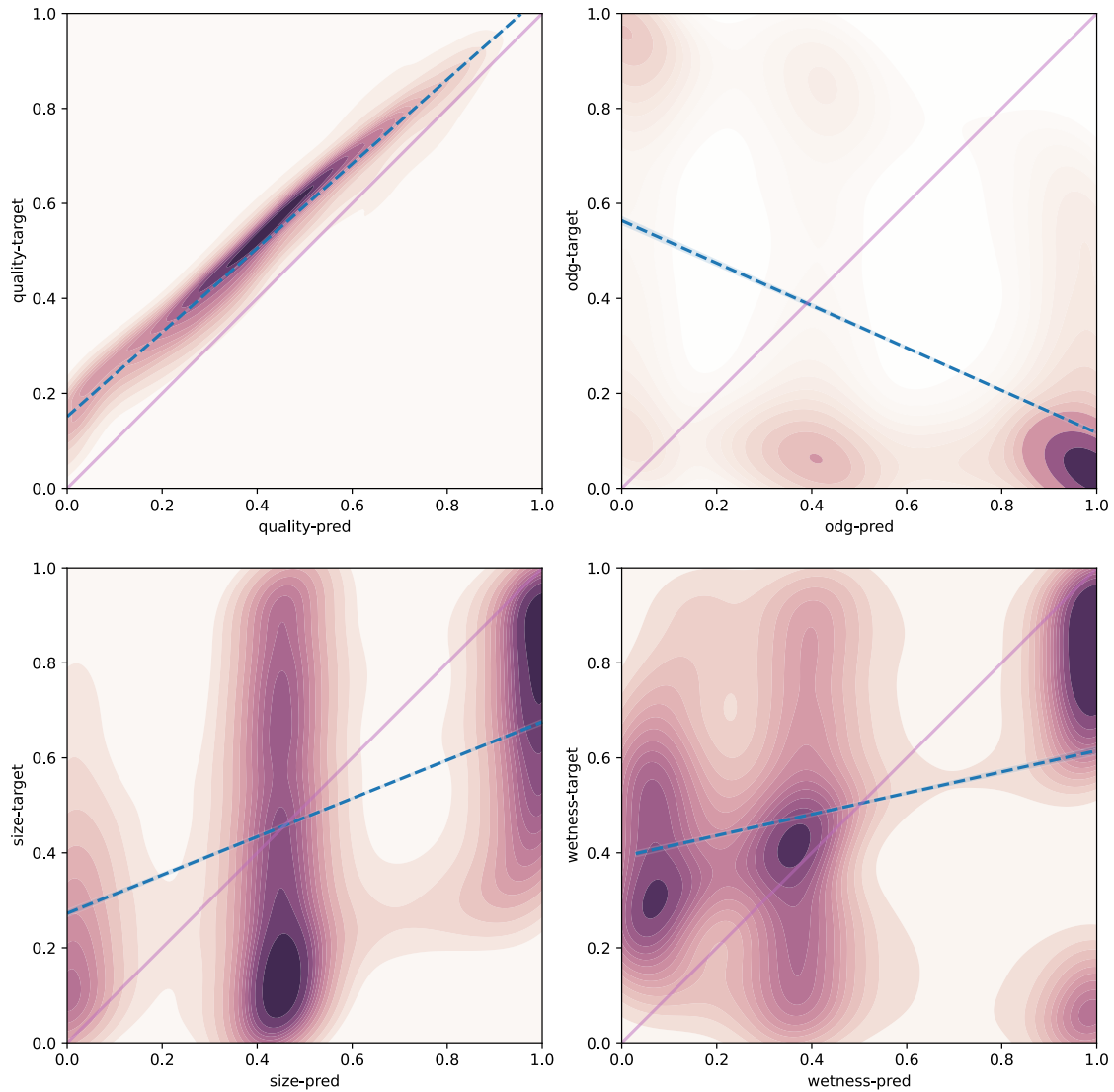


Figure 21:

TOTAL NUMBER OF SAMPLES: 2896664400 Total length of audio coded in 44.1 kHz is 18.2455555556 hours TOTAL LENGTH OF INFERENCE TIME: 7.738234307016683

6.5 Dereverberation Network

Leo Kling

which versions do the want to show the results of? only the best one (then compare si-snr with perceptual?)

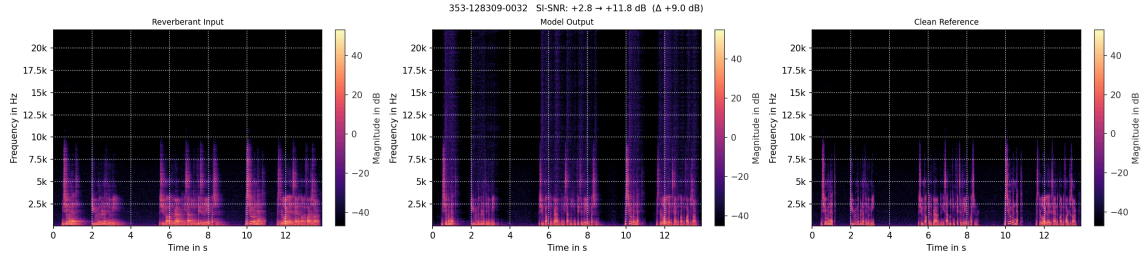


Figure 22:

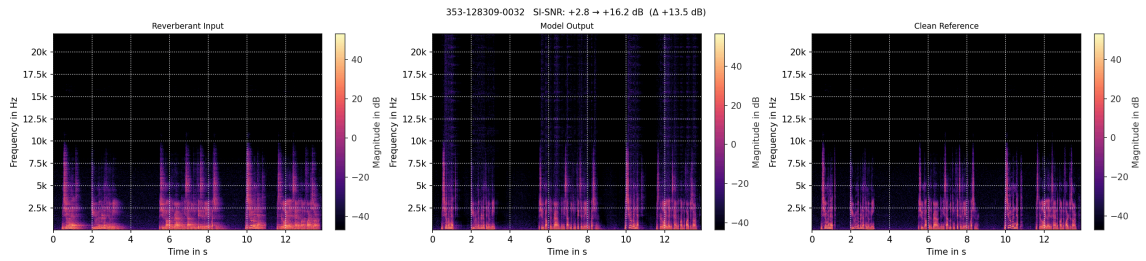


Figure 23:

- one can see the effect of the additional 39 epochs of training when comparing Figure 22 and Figure 23. While the reverberation tail is only slightly further reduced in magnitude, the high-frequency noise is substantially reduced, but still present. This is reflected in the SI-SNR improvement, which increases from 11.8 dB to 16.2 dB for this example.
- gating effect
- adds highs in some examples
 - makes sense because of “learning” at 44.1 kHz (Section 4.1.2.1)
 - “upsampling” effect almost pleasant in some cases, but also adds unwanted artifacts in others
 - discussion: maybe reverberating at 44.1 kHz would have made sense (Section 7.4)
- if the input signal is not so reverberant, the output only gets dereverberated very little
- quality of dereverberation is highly dependent on the quality of the input signal

measure inference time

7 Evaluation

7.1 Perceptual Quality Net

Jonathan Kron

Evaluation of the perceptual quality network is done both for the initial simple implementation as well as the CNN14 based one. The evaluation of the simple CNN (see Section 7.1.1) focuses on the comparison of the two approaches, examining the improvement the CNN14 based implementation made. Section 7.1.2 discusses general applicability of the perceptual quality network as loss function and its predictive performance compared to other metrics like the SI-SNR.

7.1.1 Simple CNN

This evaluation was carried out using a subset of the dataset discussed in Section 4.1.1. Both the simple CNN and the CNN14 based implementation were trained on the same subset. Training was done over 10 epochs saving only the best epoch of the training process.

The output of all prediction heads are compared to their ground truth counterpart using MSE, MAE and correlation as comparative metrics. Table 10 shows absolute differences between ground truth and prediction as well as relative improvements between the simple CNN and the CNN14 based implementation.

Metric	Simple CNN	CNN14	Relative improvement
MSE	0.00867	0.00587	32.31 %
MAE	0.04722	0.03964	16.06 %
Correlation	0.86715	0.91155	5.12 %

Table 10: Metric comparison of the quality score, taken from the best epoch of the first 10, between the simple CNN and CNN14 implementation

Table 10 shows improvements over all comparative metrics, Similar advancements have been made across all parameters (cf. Table 11).

Metric	quality score	ODG	size	wetness
MSE	32.31 %	36.19 %	35.97 %	36.07 %
MAE	16.06 %	14.6 %	25.28 %	25.32 %
Correlation	5.12 %	4.06 %	6.71 %	6.85 %

Table 11: Relative improvement from the CNN14 implementation as compared to the simple CNN implementation over all metrics and parameters, taken from the best epoch of the first 10

These findings motivated us to proceed with the CNN14 based implementation as described in Section 5.2.2 for all further experiments.

7.1.2 CNN14

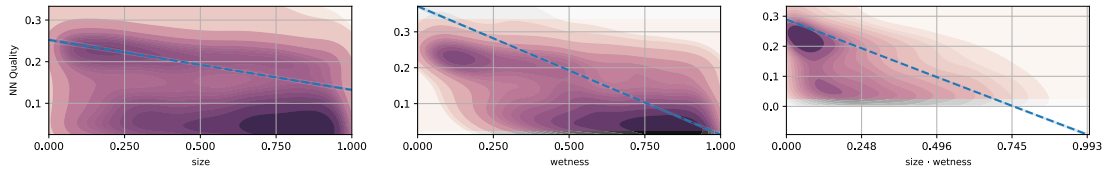


Figure 24: Quality score prediction analyzed over 16421 datapoints from test dataset (cf. Table 5), data between the 15th and 85th percentile is shown in color

- ReLU not entirely differentiable

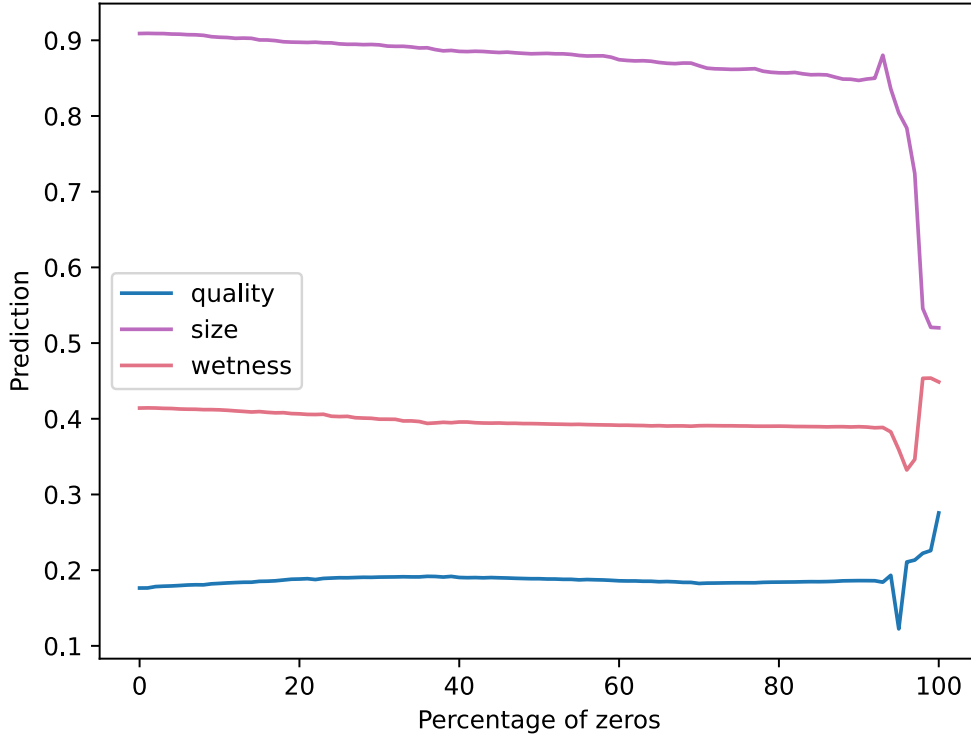


Figure 25: Prediction quality of perceptual net from signal with increasing zero percentage

7.2 Objective Quality Net

7.3 Comparison of Conv-TasNet and StoRM for diverse signals

Leo Kling

Both Conv-TasNet and StoRM were trained exclusively on speech recordings and have no exposure to music, environmental noise, or other non-speech content. To assess how each model generalises outside this group, both were applied to 2048 randomly sampled AudioSet clips spanning a wide range of acoustic scenes and event categories. Table 12 summarises these metrics, while the full distributions are shown in Figure 26.

Network	StoRM	Conv-TasNet
MSE	0.02996 ± 0.048	0.02795 ± 0.044
SI-SNR (dB)	-31.39 ± 11.71	-31.26 ± 18.6
PESQ-WB	1.35 ± 0.29	1.45 ± 0.44
PESQ-NB	1.82 ± 0.4	1.81 ± 0.64
ODG	-3.67 ± 0.48	-3.83 ± 0.14
DI	-3.14 ± 1.06	-3.5 ± 0.59
Runtime	6:14:51	0:04:15

Table 12: Comparison of different metrics for the evaluation of dereverberation performance of diverse audio samples, evaluated on 2048 random AudioSet samples. The runtime is measured on a single H100 GPU (CLAIX-2023-ML) [CLA26]) with 2048 Random AudioSet Samples.

Across all metrics both models perform substantially below their in-domain speech statistics. PESQ-WB reaches only 1.35 (StoRM) and 1.45 (Conv-TasNet), far below StoRM’s in-domain speech result of 2.83 (Table 9). ODG values of -3.67 and -3.83 place both models near the lower end of the five-step degradation scale, indicating consistently “annoying” to “very annoying” perceived quality. The boxplots in Figure 26 confirm that these results are not driven by a few extreme samples: distributions are broad but consistent, with no single extreme point pulling results in one direction. Boxplots for SI-SNR and PESQ show some narrower interquartile ranges and shorter whiskers for the StoRM model. A recurring informal observation from listening tests and viewing spectrograms is that both models tend to lower the output level relative to the input, especially for non-speech signals. This unintended effect is visible in the spectrograms (Figure 17, Figure 18) and likely contributes to the degraded metric values.

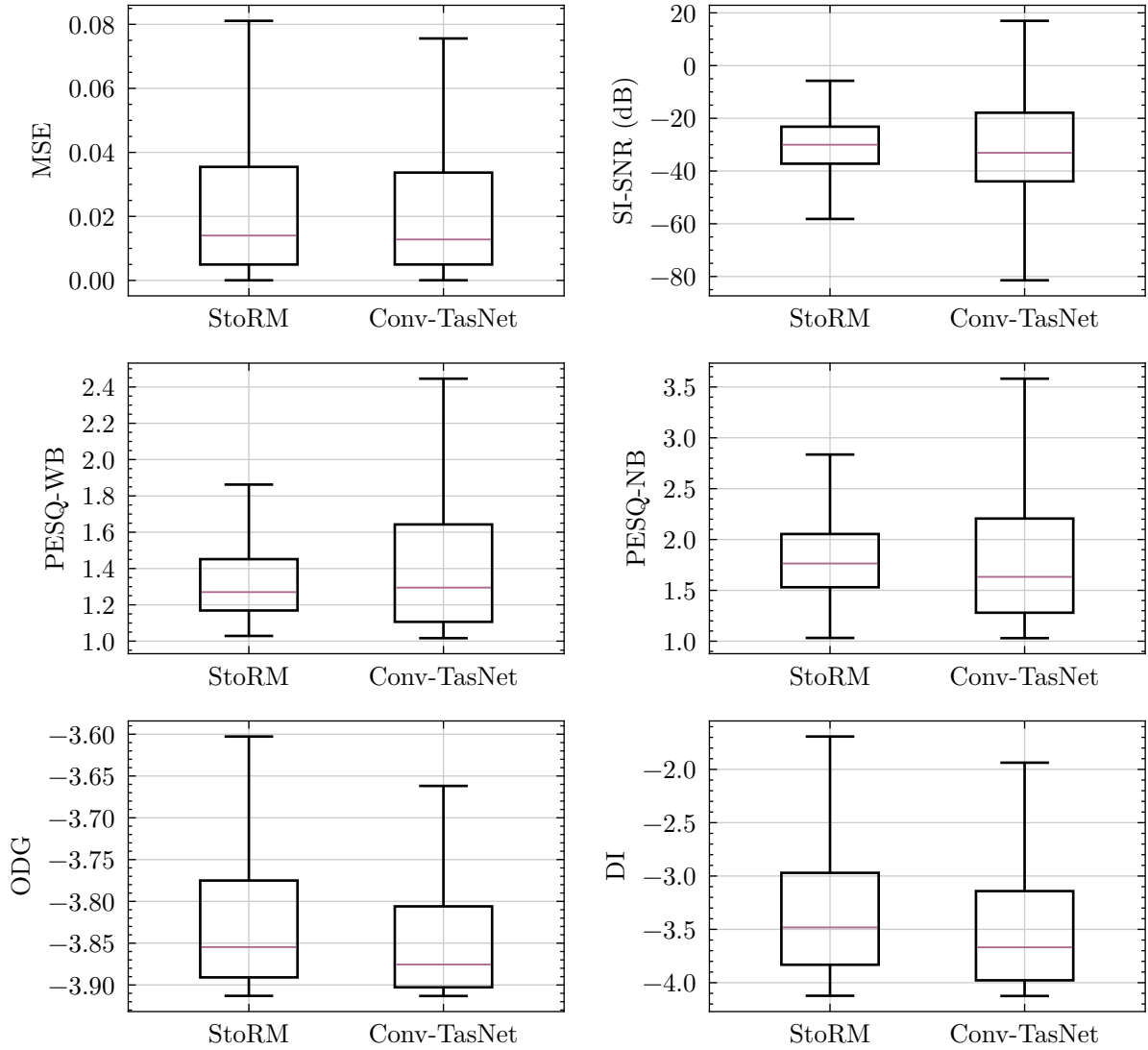


Figure 26: Boxplot comparison of different metrics for the evaluation of dereverberation performance of diverse audio samples. (Outliers not shown)

Think about outliers in boxplots (only show some?)

When comparing the two models against each other, the differences across most metrics are small relative to the standard deviation. Conv-TasNet achieves a marginally lower MSE (0.028 vs. 0.030) and slightly higher PESQ-WB (1.45 vs. 1.35), while StoRM scores better to a slight extent on ODG (-3.67 vs. -3.83) and DI (-3.14 vs. -3.50), suggesting it introduces fewer perceptual artifacts per sample on average. PESQ-NB is effectively equal (1.82 vs. 1.81).

talk about SI-SNR, only metric left undiscussed

The most unambiguous differentiator is computational cost: at comparable out-of-domain performance, Conv-TasNet processes all 2048 samples in 4 m 15 s, while StoRM requires 6 h 14 m 51 s — approximately $88 \times$ the inference time.

7.4 DISCUSS UPSAMPLING FOR TRAINING AND REVERBERATING AT LOWER (USING PLOTS)

reverberation was made in native sample rate, then upsampled for training: meaning that some files lack proper wide band reverberation and might “confuse” model

7.5 (An)echoic dataset

- AudioSet and FSD50K are not “dry” datasets. they contain samples that are recorded under echoic conditions.
- The model is not shown fully dereverberated sample pairs during training

7.6 SI-SNR Calculations

- Figure 15 and Table 12
- did we fuck up here?
- maybe thats why training Conv-TasNet with SI-SNR loss did not work as expected

8 Conclusion

List of Figures

Figure 1	Proposed Quality-Net for end-to-end, non-intrusive speech quality assessment	8
Figure 2	RNN architecture and unfolded version	12
Figure 3	TCN residual block	13
Figure 4	Dilated causal convolution	14
Figure 5	DAE training procedure	15
Figure 6		16
Figure 7	A taxonomy of loss functions	22
Figure 8	Structure of PESQ model	25
Figure 9	High-level representation of the PEAQ model	26
Figure 10	Dereverberation preprocessing pipeline	29
Figure 11	Perceptual loss preprocessing pipeline	30
Figure 12	Signal with non-silent mask	33
Figure 13	Metrics usable as loss functions	35
Figure 14	Forward pass of the dereverberation model.	45
Figure 15	Training curves for Conv-TasNet with different loss functions	47
Figure 16	Training curve for Conv-TasNet with MSS loss	48
Figure 17	Spectrogram comparison of input and output of the MSE-trained Conv-TasNet	49
Figure 18	Spectrogram comparison of input and output of StoRM	51
Figure 19		52
Figure 20		53
Figure 21		54
Figure 22		55
Figure 23		55
Figure 24	Quality score prediction analyzed over test dataset	57

Figure 25 Prediction quality of perceptual net from signal with increasing zero percentage	58
Figure 26 Boxplot comparison for dereverberation performance of diverse audio samples	60

List of Tables

Table 1	Results of Quality-Net and the two-stage model	9
Table 2	Interpretation of the Pearson coefficient	24
Table 3	The Absolute Category Rating scale used by MOS/PESQ	25
Table 4	Dataset composition	28
Table 5	Subset composition	29
Table 6	Architecture of the initial implementation of the perceptual quality network	42
Table 7	Architecture of the CNN14 based implementation of the perceptual quality network	44
Table 8	Conv-TasNet dereverberation metrics on LibriSpeech	49
Table 9	StoRM dereverberation metrics on the WSJ0+REVERB test set ...	50
Table 10	Metric comparison of the quality score, taken from the best epoch of the first 10, between the simple CNN and CNN14 implementation .	56
Table 11	Relative improvement from the CNN14 implementation as compared to the simple CNN implementation over all metrics and parameters, taken from the best epoch of the first 10	57
Table 12	Comparison of different metrics for the evaluation of dereverberation performance of diverse audio samples, evaluated on 2048 random AudioSet samples. The runtime is measured on a single H100 GPU (CLAIX-2023-ML) [CLA26]) with 2048 Random AudioSet Samples.	59

Appendix A: Supplementary Material

– Supplementary Material –

Bibliography

- [Kuh25] C. Kuhn-Rahloff, *Schall, Raum und auditive Wahrnehmung: Eine Einführung in die Grundlagen der Schallausbreitung und der Raumakustik mit Praxisbeispielen und Übungen*. Wiesbaden: Springer Fachmedien, 2025. doi: 10.1007/978-3-658-46280-2.
- [TM16] J. Traer and J. H. McDermott, “Statistics of Natural Reverberation Enable Perceptual Separation of Sound and Space,” *Proceedings of the National Academy of Sciences*, vol. 113, no. 48, pp. E7856–E7865, Nov. 2016, doi: 10.1073/pnas.1612524113.
- [Nay14] P. A. Naylor, “Chapter 31 - Dereverberation,” *Academic Press Library in Signal Processing: Volume 4*, vol. 4. in Academic Press Library in Signal Processing, vol. 4. Elsevier, pp. 879–914, 2014. doi: 10.1016/B978-0-12-396501-1.00031-5.
- [BC82] P. Bloom and G. Cain, “Evaluation of Two-Input Speech Dereverberation Techniques,” in *ICASSP '82. IEEE International Conference on Acoustics, Speech, and Signal Processing*, May 1982, pp. 164–167. doi: 10.1109/ICASSP.1982.1171717.
- [ABB77] J. B. Allen, D. A. Berkley, and J. Blauert, “Multimicrophone Signal-Processing Technique to Remove Room Reverberation from Speech Signals,” *The Journal of the Acoustical Society of America*, vol. 62, no. 4, pp. 912–915, Oct. 1977, doi: 10.1121/1.381621.
- [Fig+25] V. Figarola, W. Liang, S. Luthra, C. Brown, and B. Shinn-Cunningham, “Reverberation Exacerbates Effects of Interruption on Auditory Spatial Selective Attention,” *bioRxiv : the preprint server for biology*, 2025, doi: 10.64898/2025.12.15.694434.
- [CLG22] R. Cueille, M. Lavandier, and N. Grimault, “Effects of Reverberation on Speech Intelligibility in Noise for Hearing-Impaired Listeners,” *Royal Society Open Science*, vol. 9, no. 8, p. 210342, Aug. 2022, doi: 10.1098/rsos.210342.

- [Neu+10] A. C. Neuman, M. Wroblewski, J. Hajicek, and A. Rubinstein, “Combined Effects of Noise and Reverberation on Speech Recognition Performance of Normal-Hearing Children and Adults,” *Ear and Hearing*, vol. 31, no. 3, p. 336, June 2010, doi: 10.1097/AUD.0b013e3181d3d514.
- [Pug+21] G. E. Puglisi, A. Warzybok, A. Astolfi, and B. Kollmeier, “Effect of Reverberation and Noise Type on Speech Intelligibility in Real Complex Acoustic Scenarios,” *Building and Environment*, vol. 204, p. 108137, Oct. 2021, doi: 10.1016/j.buildenv.2021.108137.
- [Att+00] H. Attias, J. Platt, A. Acero, and L. Deng, “Speech Denoising and Dereverberation Using Probabilistic Models,” in *Advances in Neural Information Processing Systems*, MIT Press, 2000. Accessed: Dec. 14, 2025. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2000/hash/65699726a3c601b9f31bf04019c8593c-Abstract.html
- [Dat97] J. Dattorro, “Effect Design: Part 1—Reverberator and Other Filters,” *Journal of the Audio Engineering Society*, vol. 45, no. 9, pp. 660–684, Sept. 1997.
- [Bra98] M. Brandstein, “On the Use of Explicit Speech Modeling in Microphone Array Applications,” in *Proceedings of the 1998 IEEE International Conference on Acoustics, Speech and Signal Processing, ICASSP '98 (Cat. No.98CH36181)*, Seattle, WA, USA: IEEE, 1998, pp. 3613–3616. doi: 10.1109/ICASSP.1998.679662.
- [LM18] Y. Luo and N. Mesgarani, “TasNet: Time-Domain Audio Separation Network for Real-Time, Single-Channel Speech Separation,” Apr. 2018, doi: 10.48550/arXiv.1711.00541.
- [Rad21] E. Radkoff, “Loss Functions in Audio ML.” Accessed: Jan. 30, 2026. [Online]. Available: <https://soundsandwords.io/audio-loss-functions/>
- [LM19] Y. Luo and N. Mesgarani, “Conv-TasNet: Surpassing Ideal Time-Frequency Magnitude Masking for Speech Separation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 27, no. 8, pp. 1256–1266, Aug. 2019, doi: 10.1109/TASLP.2019.2915167.

- [Lem+23] J.-M. Lemerrier, J. Richter, S. Welker, and T. Gerkmann, “StoRM: A Diffusion-based Stochastic Regeneration Model for Speech Enhancement and Dereverberation,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 31, pp. 2724–2737, 2023, doi: 10.1109/TASLP.2023.3294692.
- [Sch+24] A. Schmid, M. Ambros, J. Bogon, and R. Wimmer, “Measuring the Just Noticeable Difference for Audio Latency,” in *Audio Mostly 2024 - Explorations in Sonic Cultures*, Milan Italy: ACM, Sept. 2024, pp. 325–331. doi: 10.1145/3678299.3678331.
- [Bol79] S. Boll, “Suppression of Acoustic Noise in Speech Using Spectral Subtraction,” *IEEE Transactions on Acoustics, Speech, and Signal Processing*, vol. 27, no. 2, pp. 113–120, Apr. 1979, doi: 10.1109/TASSP.1979.1163209.
- [Mad+13] N. Madhu, A. Spriet, S. Jansen, R. Koning, and J. Wouters, “The Potential for Speech Intelligibility Improvement Using the Ideal Binary Mask and the Ideal Wiener Filter in Single Channel Noise Reduction Systems: Application to Auditory Prostheses,” *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 21, no. 1, pp. 63–72, Jan. 2013, doi: 10.1109/TASL.2012.2213248.
- [Zha+21] X. Zhang *et al.*, “Low-Delay Speech Enhancement Using Perceptually Motivated Target and Loss,” in *Interspeech 2021*, ISCA, Aug. 2021, pp. 2826–2830. doi: 10.21437/Interspeech.2021-1410.
- [21] S.-W. Fu *et al.*, “MetricGAN+: An Improved Version of MetricGAN for Speech Enhancement,” 2021, doi: 10.48550/ARXIV.2104.03538.
- [22] S.-w. Fu, Y. Tsao, H.-T. Hwang, and H.-M. Wang, “Quality-Net: An End-to-End Non-intrusive Speech Quality Assessment Model Based on BLSTM,” in *Interspeech 2018*, ISCA, Sept. 2018, pp. 1873–1877. doi: 10.21437/Interspeech.2018-1802.

- [Sha49] C. Shannon, “Communication in the Presence of Noise,” *Proceedings of the IRE*, vol. 37, no. 1, pp. 10–21, Jan. 1949, doi: 10.1109/JRPROC.1949.232969.
- [Aco23] “Acoustics — Normal Equal-Loudness-Level Contours.” Accessed: Nov. 03, 2026. [Online]. Available: <https://www.iso.org/standard/83117.html>
- [Zwi61] E. Zwicker, “Subdivision of the Audible Frequency Range into Critical Bands (Frequenzgruppen),” *The Journal of the Acoustical Society of America*, vol. 33, no. 2, p. 248, Feb. 1961, doi: 10.1121/1.1908630.
- [Sch+22] H. Schröter, A. N. Escalante-B, T. Rosenkranz, and A. Maier, “Deep-FilterNet: A Low Complexity Speech Enhancement Framework for Full-Band Audio Based on Deep Filtering,” Feb. 2022, doi: 10.48550/arXiv.2110.05588.
- [MH20] W. Mack and E. A. P. Habets, “Deep Filtering: Signal Extraction and Reconstruction Using Complex Time-Frequency Filters,” *IEEE Signal Processing Letters*, vol. 27, pp. 61–65, 2020, doi: 10.1109/LSP.2019.2955818.
- [Gar+93] Garofolo, John S. *et al.*, *TIMIT Acoustic-Phonetic Continuous Speech Corpus*. (1993). Linguistic Data Consortium. doi: 10.35111/17GK-BN40.
- [Sid20] S. Siddiq, “Optimization of Convolution Reverberation,” presented at the International Conference on Digital Audio Effects (DAFx), 2020. Accessed: Mar. 05, 2026. [Online]. Available: https://www.dafx.de/paper-archive/details/Rj5F378RFBW38OU_AcbnDA
- [MNT16] M. Misić, D. Nikolov, and M. Tomasevic, “Analysis of CPU and GPU Implementations of Convolution Reverb Effect,” *Telfor Journal*, vol. 8, no. 2, pp. 121–126, 2016, doi: 10.5937/telfor1602121M.
- [GBC16] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

- [Gra+25] J. Grau-Haro, R. Ribes-Serrano, J. Naranjo-Alcazar, M. Garcia-Balles-
teros, and P. Zuccarello, “Comprehensive Evaluation of CNN-Based
Audio Tagging Models on Resource-Constrained Devices,” 2025, doi:
10.48550/ARXIV.2509.14049.
- [Kon+20] Q. Kong, Y. Cao, T. Iqbal, Y. Wang, W. Wang, and M. D. Plumbley,
“PANNs: Large-Scale Pretrained Audio Neural Networks for Audio
Pattern Recognition,” Aug. 2020, doi: 10.48550/arXiv.1912.10211.
- [RHW86] D. E. Rumelhart, G. E. Hinton, and R. J. Williams, “Learning Repre-
sentations by Back-Propagating Errors,” *Nature*, vol. 323, no. 6088,
pp. 533–536, Oct. 1986, doi: 10.1038/323533a0.
- [HS97] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,”
Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi:
10.1162/neco.1997.9.8.1735.
- [Cho+14] K. Cho, B. van Merriënboer, D. Bahdanau, and Y. Bengio, “On
the Properties of Neural Machine Translation: Encoder–Decoder Ap-
proaches,” in *Proceedings of SSST-8, Eighth Workshop on Syntax, Se-
mantics and Structure in Statistical Translation*, D. Wu, M. Carpuat,
X. Carreras, and E. M. Vecchi, Eds., Doha, Qatar: Association for
Computational Linguistics, Oct. 2014, pp. 103–111. doi: 10.3115/v1/
W14-4012.
- [Déf+19] A. Défossez, N. Usunier, L. Bottou, and F. Bach, “Music
Source Separation in the Waveform Domain,” 2019, doi: 10.48550/
ARXIV.1911.13254.
- [BKK18] S. Bai, J. Z. Kolter, and V. Koltun, “An Empirical Evaluation of
Generic Convolutional and Recurrent Networks for Sequence Model-
ing,” Apr. 2018, doi: 10.48550/arXiv.1803.01271.
- [39] K. Lee, J. Ray, and C. Safta, “The Predictive Skill of Convolutional
Neural Networks Models for Disease Forecasting,” *PLOS ONE*, vol.
16, p. e254319, July 2021, doi: 10.1371/journal.pone.0254319.

- [HZ93] G. E. Hinton and R. S. Zemel, “Autoencoders, Minimum Description Length and Helmholtz Free Energy,” in *Proceedings of the 7th International Conference on Neural Information Processing Systems*, in NIPS'93. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., Nov. 1993, pp. 3–10.
- [LF87] Y. Le Cun and F. Fogelman-Soulié, “Modèles connexionnistes de l'apprentissage,” *Intellectica*, vol. 2, no. 1, pp. 114–143, 1987, doi: 10.3406/intel.1987.1804.
- [42] A. Baevski, H. Zhou, A. Mohamed, and M. Auli, “Wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations,” Oct. 2020, doi: 10.48550/arXiv.2006.11477.
- [Wid60] B. Widrow, “An Adaptive ‘Adaline’ Neuron Using Chemical ‘Memistors’,” 1553–2, Oct. 1960. [Online]. Available: <https://www-isl.stanford.edu/~widrow/papers/t1960anadaptive.pdf>
- [Ama93] S.-i. Amari, “Backpropagation and Stochastic Gradient Descent Method,” *Neurocomputing*, vol. 5, no. 4–5, pp. 185–196, June 1993, doi: 10.1016/0925-2312(93)90006-O.
- [Bay+15] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic Differentiation in Machine Learning: A Survey,” 2015, doi: 10.48550/ARXIV.1502.05767.
- [Aut26] “Autograd Mechanics — PyTorch 2.10 Documentation.” Accessed: Mar. 18, 2026. [Online]. Available: <https://docs.pytorch.org/docs/stable/notes/autograd.html>
- [Cia+24] L. Ciampiconi, A. Elwood, M. Leonardi, A. Mohamed, and A. Rozza, “A Survey and Taxonomy of Loss Functions in Machine Learning,” Nov. 2024, doi: 10.48550/arXiv.2301.05579.
- [SED18] D. Stoller, S. Ewert, and S. Dixon, “Wave-U-Net: A Multi-Scale Neural Network for End-to-End Audio Source Separation,” 2018, doi: 10.48550/ARXIV.1806.03185.

- [TM20] N. Takahashi and Y. Mitsufuji, “D3Net: Densely Connected Multidilated DenseNet for Music Source Separation,” 2020, doi: 10.48550/ARXIV.2010.01733.
- [Ben+09] J. Benesty, J. Chen, Y. Huang, and I. Cohen, “Pearson Correlation Coefficient,” *Noise Reduction in Speech Processing*. Springer, Berlin, Heidelberg, pp. 1–4, 2009. doi: 10.1007/978-3-642-00296-0_5.
- [Cor+19] H. Cordourier, P. Lopez Meyer, J. Huang, J. Del Hoyo Ontiveros, and H. Lu, “GCC-PHAT Cross-Correlation Audio Features for Simultaneous Sound Event Localization and Detection (SELD) on Multiple Rooms,” in *Proceedings of the Detection and Classification of Acoustic Scenes and Events 2019 Workshop (DCASE2019)*, New York University, 2019, pp. 55–58. doi: 10.33682/3re4-nd65.
- [VGF06] E. Vincent, R. Gribonval, and C. Fevotte, “Performance Measurement in Blind Audio Source Separation,” *IEEE Transactions on Audio, Speech and Language Processing*, vol. 14, no. 4, pp. 1462–1469, July 2006, doi: 10.1109/TSA.2005.858005.
- [Rix+01] A. Rix, J. Beerends, M. Hollier, and A. Hekstra, “Perceptual Evaluation of Speech Quality (PESQ)-a New Method for Speech Quality Assessment of Telephone Networks and Codecs,” in *2001 IEEE International Conference on Acoustics, Speech, and Signal Processing. Proceedings (Cat. No.01CH37221)*, Salt Lake City, UT, USA: IEEE, 2001, pp. 749–752. doi: 10.1109/ICASSP.2001.941023.
- [Thi+00] T. Thiede *et al.*, “PEAQ - the ITU Standard for Objective Measurement of Perceived Audio Quality,” *Journal of the Audio Engineering Society*, 2000, Accessed: Mar. 11, 2026. [Online]. Available: <https://publica.fraunhofer.de/handle/publica/196639>
- [KAD17] A. F. Khalifeh, A.-K. Al-Tamimi, and K. A. Darabkh, “Perceptual Evaluation of Audio Quality under Lossy Networks,” in *2017 International Conference on Wireless Communications, Signal Processing and*

- Networking (WiSPNET)*, Chennai: IEEE, Mar. 2017, pp. 939–943. doi: 10.1109/WiSPNET.2017.8299900.
- [Den+09] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and F.-F. Li, “ImageNet: A Large-Scale Hierarchical Image Database,” presented at the IEEE Conference on Computer Vision and Pattern Recognition, June 2009, pp. 248–255. doi: 10.1109/CVPR.2009.5206848.
- [Nav+25] H. Naveed *et al.*, “A Comprehensive Overview of Large Language Models,” *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 5, pp. 106:1–106:72, Aug. 2025, doi: 10.1145/3744746.
- [Gar+07] Garofolo, John S., Graff, David, Paul, Doug, and Pallett, David, *CSR-I (WSJ0) Complete*. (May 30, 2007). Linguistic Data Consortium. doi: 10.35111/EWKM-CG47.
- [Pan+15] V. Panayotov, G. Chen, D. Povey, and S. Khudanpur, “Librispeech: An ASR Corpus Based on Public Domain Audio Books,” in *2015 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2015, pp. 5206–5210. doi: 10.1109/ICASSP.2015.7178964.
- [Ric+24] J. Richter *et al.*, “EARS: An Anechoic Fullband Speech Dataset Benchmarked for Speech Enhancement and Dereverberation,” June 2024, doi: 10.48550/arXiv.2406.06185.
- [Woo92] J. Woodard, “Modeling and Classification of Natural Sounds by Product Code Hidden Markov Models,” *Trans. Sig. Proc.*, vol. 40, no. 7, pp. 1833–1835, July 1992, doi: 10.1109/78.143457.
- [Ell01] D. P. W. Ellis, “Detecting Alarm Sounds,” 2001, doi: 10.7916/D8ZS35XZ.
- [Mes+24] A. Mesaros, R. Serizel, T. Heittola, T. Virtanen, and M. D. Plumbley, “A Decade of DCASE: Achievements, Practices, Evaluations and Future Challenges,” Oct. 2024, doi: 10.48550/arXiv.2410.04951.

- [Smi10] J. O. I. Smith, *Physical Audio Signal Processing: For Virtual Musical Instruments and Audio Effects*. Stanford, Calif: Stanford University, CCRMA, 2010.
- [Gem+17] J. F. Gemmeke *et al.*, “Audio Set: An Ontology and Human-Labeled Dataset for Audio Events,” in *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Mar. 2017, pp. 776–780. doi: 10.1109/ICASSP.2017.7952261.
- [Fon+22] E. Fonseca, X. Favory, J. Pons, F. Font, and X. Serra, “FSD50K: An Open Dataset of Human-Labeled Sound Events,” *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 30, pp. 829–852, 2022, doi: 10.1109/TASLP.2021.3133208.
- [Riv92] R. Rivest, “The MD5 Message-Digest Algorithm.” [Online]. Available: <https://www.ietf.org/rfc/rfc1321.txt>
- [JSV09] M. Jeub, M. Schäfer, and P. Vary, “A Binaural Room Impulse Response Database for the Evaluation of Dereverberation Algorithms,” in *Proceedings of International Conference on Digital Signal Processing (DSP)*, Santorini, Greece: IEEE / IEEE, IET, EURASIP, July 2009, pp. 1–4.
- [Sch18] S. Schlecht, “Feedback Delay Networks in Artificial Reverberation and Reverberation Enhancement,” 2018.
- [Far07] A. Farina, “Impulse Response Measurements,” in *23rd Nordic Sound Symposium*, Bolkesjø, Norway, 2007, pp. 27–30.
- [Man+25] J. Mannall, L. Savioja, A. Neidhardt, R. Mason, and E. Sena, *RoomAcoustiC++: An Open-Source Room Acoustic Model for Real-Time Audio Simulations*. 2025.
- [SBD18] R. Scheibler, E. Bezzam, and I. Dokmanić, “Pyroomacoustics: A Python Package for Audio Room Simulations and Array Processing Algorithms,” in *2018 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, Apr. 2018, pp. 351–355. doi: 10.1109/ICASSP.2018.8461310.

- [AB79] J. Allen and D. Berkley, “Image Method for Efficiently Simulating Small-Room Acoustics,” *The Journal of the Acoustical Society of America*, vol. 65, pp. 943–950, Apr. 1979, doi: 10.1121/1.382599.
- [Vor08] M. Vorländer, *Auralization: Fundamentals of Acoustics, Modelling, Simulation, Algorithms and Acoustic Virtual Reality*, 1st ed. Berlin: Springer, 2008.
- [Sob23] P. Sobot, “Pedalboard.” Apr. 10, 2023. doi: 10.5281/ZENODO.7817839.
- [Pu06] I. M. Pu, “Audio Compression,” *Fundamental Data Compression*. Elsevier, pp. 171–188, 2006. doi: 10.1016/B978-075066310-6/50012-X.
- [ITU88] ITU-T, “G.711: Pulse Code Modulation (PCM) of Voice Frequencies,” Recommendation G.711, 1988. [Online]. Available: <https://www.itu.int/rec/T-REC-G.711>
- [HZ15] M. Holters and U. Zölzer, “GstPEAQ – an Open Source Implementation of the PEAQ Algorithm,” 2015.
- [GSt26] “GStreamer/Gst-Python.” Accessed: Mar. 06, 2026. [Online]. Available: <https://gitlab.freedesktop.org/gstreamer/gstreamer/-/tree/main/subprojects/gst-python>
- [Rob26] J. Robert, “Jiaaro/Pydub.” Accessed: Mar. 10, 2026. [Online]. Available: <https://github.com/jiaaro/pydub>
- [Chi+20] M. Chinen, F. S. C. Lim, J. Skoglund, N. Gureev, F. O’Gorman, and A. Hines, “ViSQOL v3: An Open Source Production Ready Objective Speech and Audio Metric,” Apr. 2020, doi: 10.48550/arXiv.2004.09584.
- [HK06] R. Huber and B. Kollmeier, “PEMO-Q—A New Method for Objective Audio Quality Assessment Using a Model of Auditory Perception,” *IEEE Transactions on Audio, Speech and Language Processing*, vol. 14, no. 6, pp. 1902–1911, Nov. 2006, doi: 10.1109/TASL.2006.883259.
- [DH20] P. M. Delgado and J. Herre, “Can We Still Use PEAQ? A Performance Analysis of the ITU Standard for the Objective Assessment of

- Perceived Audio Quality,” in *2020 Twelfth International Conference on Quality of Multimedia Experience (QoMEX)*, May 2020, pp. 1–6. doi: 10.1109/QoMEX48832.2020.9123105.
- [Rat+03] R. Ratnam, D. L. Jones, B. C. Wheeler, W. D. O’Brien, C. R. Lansing, and A. S. Feng, “Blind Estimation of Reverberation Time,” *The Journal of the Acoustical Society of America*, vol. 114, no. 5, pp. 2877–2892, Nov. 2003, doi: 10.1121/1.1616578.
- [KB17] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” Jan. 2017, doi: 10.48550/arXiv.1412.6980.
- [LH19] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” Jan. 2019, doi: 10.48550/arXiv.1711.05101.
- [O’S87] D. O’Shaughnessy, *Speech Communication: Human and Machine*. in Addison-Wesley Series in Electrical Engineering. Reading, Mass: Addison-Wesley Pub. Co, 1987.
- [Sim26] Simon King, “Bark Scale vd Mel Scale.” Accessed: Mar. 16, 2026. [Online]. Available: <https://speech.zone/forums/topic/bark-scale-vd-mel-scale/>
- [Dho+19] D. S. B. Dhonde, A. A. Chaudhari, Assistant Professor, Department of Electronics & Telecommunication Engineering, A.I.S.S.M.S, Institute of Information Technology, Pune, India, M. P. Gajare, and Assistant Professor, Department of Electronics & Telecommunication Engineering, A.I.S.S.M.S, Institute of Information Technology, Pune, India, “Performance Evaluation of Mel and Bark Scale Based Features for Text-Independent Speaker Identification,” *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, no. 11, pp. 3734–3738, Sept. 2019, doi: 10.35940/ijitee.K1999.0981119.
- [Cho+17] K. Choi, G. Fazekas, K. Cho, and M. Sandler, “A Comparison of Audio Signal Preprocessing Methods for Deep Neural Networks on Music Tagging,” 2017, doi: 10.48550/ARXIV.1709.01922.

- [GB10] X. Glorot and Y. Bengio, “Understanding the Difficulty of Training Deep Feedforward Neural Networks,” *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, vol. 9. in Proceedings of Machine Learning Research, vol. 9. PMLR, pp. 249–256, May 15, 2010. [Online]. Available: <https://proceedings.mlr.press/v9/glorot10a.html>
- [92] S. Schwär and M. Müller, “Multi-Scale Spectral Loss Revisited,” *IEEE Signal Processing Letters*, vol. 30, pp. 1712–1716, 2023, doi: 10.1109/LSP.2023.3333205.
- [Kin+13] K. Kinoshita *et al.*, “The Reverb Challenge: A Common Evaluation Framework for Dereverberation and Recognition of Reverberant Speech,” in *IEEE Workshop on Applications of Signal Processing to Audio and Acoustics*, 2013, pp. 22–23.
- [CLA26] “CLAIX-2023 (RWTH High Performance Computing (Linux)) - IT Center Help.” Accessed: Mar. 12, 2026. [Online]. Available: <https://help.itc.rwth-aachen.de/service/rhr4fjjutttf/article/fbd107191cf14c4b8307f44f545cf68a/>